



AKADEMIA GÓRNICZO-HUTNICZA IM. STANISŁAWA STASZICA W KRAKOWIE

**WYDZIAŁ ELEKTROTECHNIKI, AUTOMATYKI,
INFORMATYKI I INŻYNIERII BIOMEDYCZNEJ**

Praca dyplomowa

Development and tests of an algorithm for style and authorship
attribution of paintings.

*Opracowanie i testowanie algorytmu do ustalania stylu i autorstwa
obrazów*

Autor:	<i>Monika Mundala</i>
Kierunek studiów:	<i>Informatyka i Systemy Inteligentne</i>
Opiekun pracy:	<i>dr inż. Piotr Szwed</i>

Kraków, 2024

Upředzony o odpowiedzialności karnej na podstawie art. 115 ust. 1 i 2 ustawy z dnia 4 lutego 1994 r. o prawie autorskim i prawach pokrewnych (t.j. Dz.U. z 2006 r. Nr 90, poz. 631 z późn. zm.): „Kto przywłaszcza sobie autorstwo albo wprowadza w błąd co do autorstwa całości lub części cudzego utworu albo artystycznego wykonania, podlega grzywnie, karze ograniczenia wolności albo pozbawienia wolności do lat 3. Tej samej karze podlega, kto rozpowszechnia bez podania nazwiska lub pseudonimu twórcy cudzy utwór w wersji oryginalnej albo w postaci opracowania, artystyczne wykonanie albo publicznie zniekształca taki utwór, artystyczne wykonanie, fonogram, wideogram lub nadanie.”, a także upředzony o odpowiedzialności dyscyplinarnej na podstawie art. 211 ust. 1 ustawy z dnia 27 lipca 2005 r. Prawo o szkolnictwie wyższym (t.j. Dz. U. z 2012 r. poz. 572, z późn. zm.) „Za naruszenie przepisów obowiązujących w uczelni oraz za czyny uchylbiające godności studenta student ponosi odpowiedzialność dyscyplinarną przed komisją dyscyplinarną albo przed sądem koleżeńskim samorządu studenckiego, zwanym dalej „sądem koleżeńskim”, oświadczam, że niniejszą pracę dyplomową wykonałem(-am) osobiście i samodzielnie i że nie korzystałem(-am) ze źródeł innych niż wymienione w pracy.

*Special thanks to my supervisor Doctor Piotr Szwed
for all the help he provided me with during the making of this work.*

Table of content

1. Introduction	5
1.1. Goals of the project	6
1.2. Structure of the work	7
2. Theoretical background	8
2.1. Convolutional Neural Network	8
2.2. Transfer learning	10
2.3. ResNet50	12
2.4. Hierarchical Classification	13
2.5. Multilabel Classification	15
2.6. Understanding and classification of art images in literature	16
2.7. Evaluation Metrics	18
3. Implementation	22
3.1. Dataset	22
3.2. Fragment extraction	26
3.3. Data augmentation	27
3.4. Recognizing painters and on this basis inferring style	28
3.5. Recognizing authors and styles simultaneously	30
3.6. Two-stage classifier that first recognizes the style and then the author	31
4. Results	32
4.1. AtS Model	32
4.2. AaS model	35
4.3. StA Model.	38
4.4. Comparison of the models	45
5. Conclusions	48
6. References	50

1. Introduction

Artistic expression has always been a fundamental aspect of human culture. Different styles and techniques allow artists to reflect various narratives through their art. Attribution of authorship and style is an essential task in the field of art history, curation, and digital archiving. Thus, such attribution is a commonly studied problem in the field of artificial intelligence. Traditionally, this classification relied on experts with extensive knowledge and their manual analysis which, while effective, can be extremely time-consuming and sometimes subjective. Moreover, even among experts, the classification of art can be fraught with difficulties due to the subtle variations in style and technique that may exist within an artist's body of work or across different art movements. With continually advancing methods in computer vision there is a possibility of automating the task with accuracy as good as that of human experts. With the significant shift toward automation of the analysis and classification of art, there has been more interest in creating models suitable for this task. However, machines face their own set of difficulties when it comes to classification. Artworks often exhibit significant intra-class variability, as opposed to other images where objects might have consistent and recognizable patterns. Artists often experiment with different techniques and styles, making it difficult to accurately attribute the work to a specific creator or style. Furthermore, art often is abstract and interpretative, which can make it challenging for the algorithms to understand and capture certain features, which could be indicative of an artist or style. Machines have limited ability to understand visualization and abstraction in paintings, making the classification a difficult task.

This thesis explores the application of deep learning to the task of artist and style classification. An approach that has not been previously used in other works concerning art was tested. The method used in this project involves a comprehensive strategy and utilizes a balanced dataset representing a few selected artists and styles, taking advantage of their hierarchical relation. Developed algorithms attempt to attribute style and authorship based on clipped fragments of paintings. The task is challenging due to the nuanced differences between artists and the stylistic movements they represent, as well as limited resources and often not the best quality of available images.

An important aspect of this project is collecting the data to ensure its distribution and hierarchical structure. The hierarchy assumes that the styles and artists within it are organized systematically and that each artist represents only one stylistic movement, simplifying the classification process. The goal is to create a balanced dataset, which will allow equal

representation for each artist and style. It is most common in literature to include all styles an artist creates, without any consideration for the style aspect. This paper aims to propose an approach that separates the artists more distinctly, by including only one style they create in.

1.1. Goals of the project

The project focuses on designing and testing three distinct models tailored to hierarchical classification:

- First, a model that identifies painters and then infers style based on that recognition. With a simplification of each artist belonging to one style, this problem becomes the most straightforward, as the model only focuses on recognizing artists.
- The second model simultaneously recognizes both painters and styles and utilizes a loss function to incorporate both labels. The multilabel approach is commonly used in classification problems.
- The third, and final model relies on first identifying the style and then identifying the painter but only from ones belonging to the determined style thus making the artist classification a bit simpler since there is a reduced set of potential painters to consider during the classification.

All the necessary classifiers were trained on randomly selected fragments of paintings of a fixed size. The data was augmented according to the most commonly used methods to ensure the robustness and generalizability of the models. A balanced test set was compiled to assess the performance of the classifiers.

Moreover, transfer learning was applied through a pre-trained deep convolutional neural network. Those models were also fine-tuned to best improve the results of the classifications. By utilizing the mentioned approaches this project aims to contribute to the field of art classification, providing a comparison of possible methods for best results.

The main goal of this project is to create all the models, perform the training and validation, and in the end compare each approach to find their pros and cons and identify the most successful one among the ones tested. With all the models, the goal is to reach the best results within a certain time frame and using the limited resources available. For the purposes of training, image data was collected from multiple sources to create a balanced dataset.

1.2. Structure of the work

This project contains three main sections. First delves into some theoretical background and includes comprehensive research into existing work related to the topic of this project. In the third chapter, the implementation is discussed. At the beginning technologies used in the project are mentioned. Then the chapter consists of an exploration of the dataset in Chapter 3.1, and fragment extraction in Chapter 3.2., as well as data augmentation techniques and pre-processing of the gathered data in Chapter 3.3. Next, chapters (Chapters 3.4-3.6) contain an explanation of all the models created in this project. Finally, in Chapter 4, a comparison of all the developed models is made. The comparison allows to point out the most accurate model out of the ones created (Chapter 4.1). At the end of the chapter, there is a discussion, along with consideration of possible future improvements (Chapter 4.2). The fifth chapter consists of a summary of the work. Below is a diagram representing the main part of the work done in this project (Fig 1.2.1).

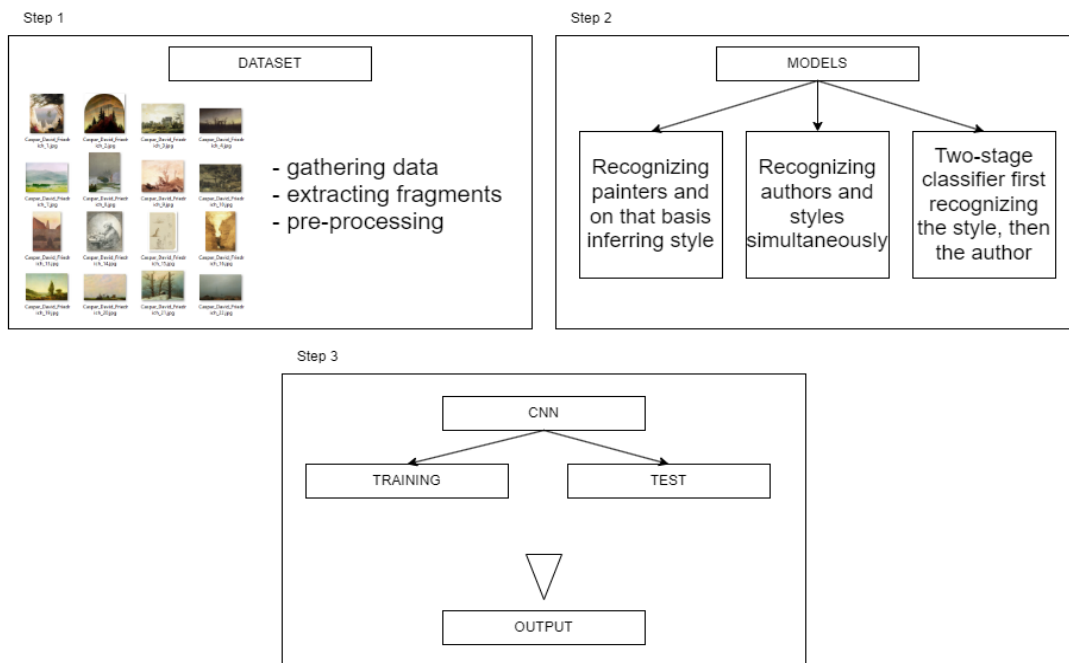


Fig 1.2.1. Diagram of the system developer in this thesis.

2. Theoretical background

This chapter will delve into the theory, necessary to explain the main concepts in this work. Concepts such as Convolutional Neural Networks, Multilabel Classification, and others will be introduced. Additionally, we will explore the hierarchical structure of the approach utilized in this work. Understanding these theoretical foundations will provide insights into how various approaches are implemented and assessed. A comprehensive literature study will also be done to introduce existing papers related to the topic of this thesis.

2.1. Convolutional Neural Network

Convolutional Neural Networks (CNNs) are artificial intelligence models based on neural networks that consist of convolutional layers. They are designed for processing visual data, like images, thus they are often used for tasks such as image analysis, object detection, image segmentation, and image classification. CNNs are also used in natural language processing tasks [1][2]. Fig. 1 shows the basic components of a CNN architecture.

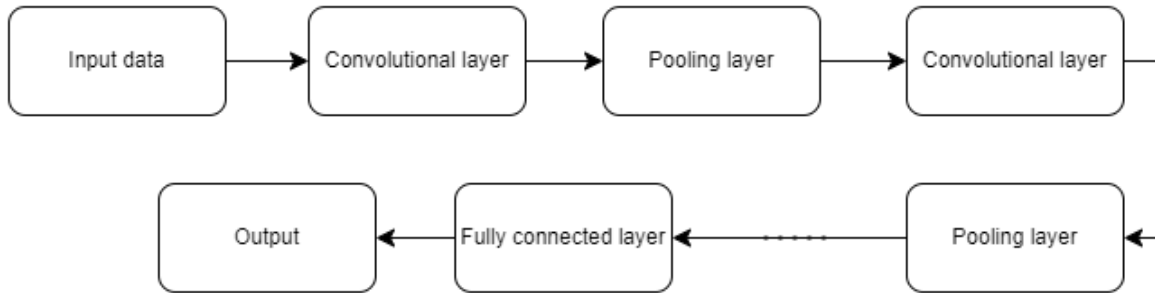


Fig 2.1.1. The basic structure of a CNN model.

Input data in the case of CNNs is usually an image dataset that is fed into the network, in most cases represented as a multi-channel matrix. Convolutional layers apply filters to the input data to detect certain features like edges, textures, or shapes. Each filter activates specific neurons, leading to the creation of feature maps. Neurons represent individual units that are activated when specific patterns are detected in the data. In the pooling layers, spatial dimensions of those maps are reduced, usually by Max or Average Pooling. This allows the model to retain the essential features while reducing the computational complexity. The fully

connected layer is responsible for making connections between each neuron from previous layers with neurons in the current layer, which integrates previously detected features. Finally, the integrated features allow the model to make the final prediction. The output layer provides the final prediction or classification, determining the most likely category for the input data [1][2][3][4].

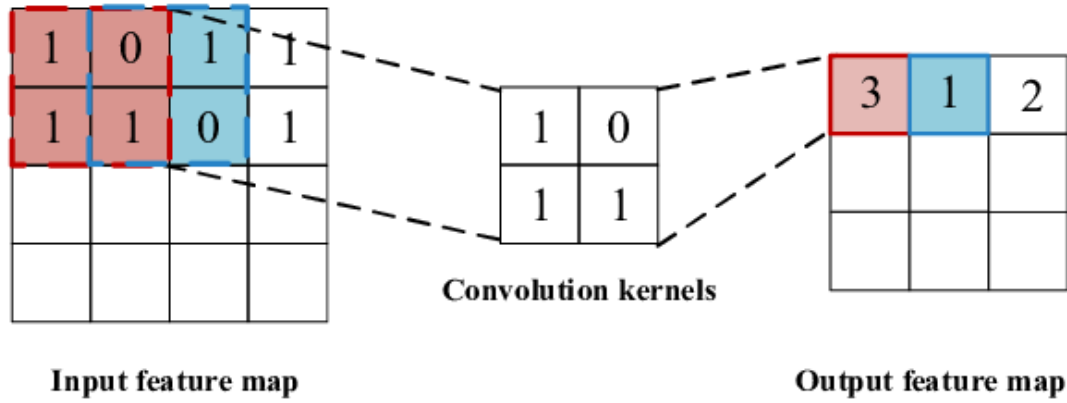


Figure 2.1.2. Diagram representing the principle of the convolution process [5].

Convolution, a fundamental operation in CNNs enables the network to automatically extract hierarchical features from input data. Its operation involves sliding a small filter (kernel) over the input image, performing element-wise multiplication between the kernel and the corresponding receptive field of the input, and summing the results. This produces a feature map, which highlights the presence of certain features of the input (edges, textures, shapes), depending on the learned weights of the kernel. Convolution reduces the number of parameters and computations by sharing weights across the image, leading to more efficient learning [6]. Figure 2.1.2 visually demonstrates the convolution process, showing how the filter moves across the input image and how the feature map is constructed from the resulting computations.

For CNNs as well as other architectures it is quite common to use a training metric, which is loss function, which quantifies the difference between predicted outputs and the true labels. One such function is Cross-Entropy loss. Its general idea is to calculate the difference between probability distributions. For binary classification, the cross-entropy between a predicted probability $\hat{y}$ and the actual label y is given by the following formula:

$$L = -[y * \log(\hat{y}) + (1 - y) * \log(1 - \hat{y})]$$

In this formula y is the ground truth label (0 or 1), and $\hat{y}$ is the predicted probability of the positive class. For multi-class problems with X classes, Cross-Entropy loss for a single data point is calculated as follows:

$$L = - \sum_{x=1}^X y_x * \log (\hat{y}_x)$$

In this formula, y_x is a binary indicator if class x is the correct class for the current observation, while $\hat{y}_x$ is the predicted probability for class x . Cross-Entropy loss shows the difference between true labels and the predicted probabilities output by the model. It quantifies how well the models' predictions match the actual distribution of the data. Lower values of the cross-entropy indicate better model performance. During training, the model's parameters (e.g., weights) are adjusted to minimize the loss, improving the model's predictions [7][8].

2.2. Transfer learning

In this project transfer learning is utilized since it is often used for image classification tasks to help achieve better performance. Transfer learning is a technique that allows adapting a pre-trained model to solve other related tasks. The advantage of using such a pre-trained model is that it is usually trained on a large dataset and can apply its learned knowledge to the current problem being studied [9]. ResNet50 (a variation of ResNet), used in this work, is a convolutional neural network that is often used in image classification. It was trained on the ImageNet dataset, which contains a large number of images from various classes. The way to adapt and utilize such a model is to remove the last fully connected layer and replace it with a layer specific to the task that is being solved. [10][11]. Figure 2.2.1. shows the general idea behind how transfer learning works.

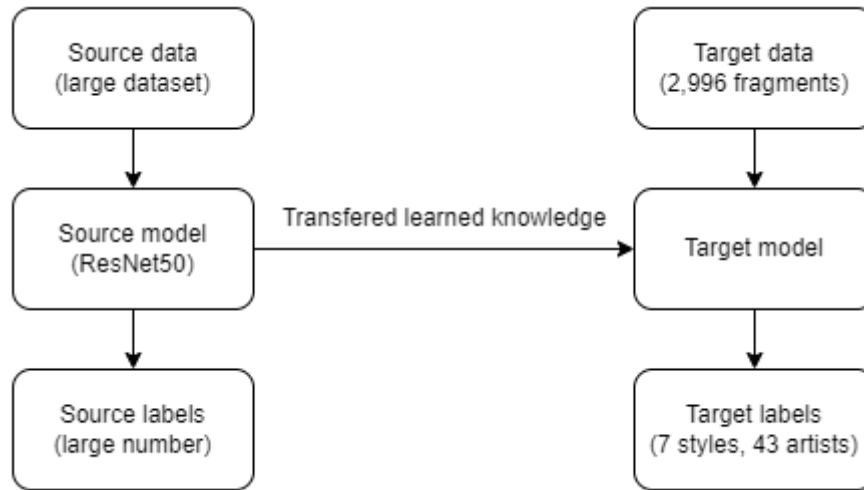


Fig 2.2.1. Transfer learning overview [12]

In the approach of using transfer learning without fine-tuning the pre-trained model is adapted to the new tasks by removing the last fully connected layer and replacing it with a new layer specific to the task at hand. The key point here is that the weights of the earlier layers are "frozen." This means that these layers do not undergo any further training or adjustment when applied to the new dataset. The newly added layers are trained, allowing the model to focus on adapting the learned representations from the pre-trained model to the specifics of the new task. This approach is especially useful when the dataset being used for the new task is small since the general features learned from the original dataset (such as edges, shapes, and textures) are preserved, reducing the risk of overfitting.

Another approach, referred to as fine-tuning entails "unfreezing" some or all of the earlier layers of the pre-trained model, allowing the weights in these layers to be updated during training on the new dataset. Fine-tuning allows the model to adjust its previously learned features to better fit the specifics of the data used. During the fine-tuning, in some cases, the earlier layers of the pre-trained model are "frozen". That means their weights are not updated during training. This allows the model to fine-tune only the later layers, which are more task-specific while retaining the general features that were learned from the original dataset. By leveraging those methods, transfer learning helps improve the efficiency of the network, while reducing the risk of overfitting [13][14].

2.3. ResNet50

ResNet50 is a Convolutional Neural Network architecture. It is one of the Residual Network (ResNet) family, which is a series of models that are created to solve tasks connected to training deep neural networks. ResNet50 is a 50-layer version of ResNet that contains convolutional layers, an identity block, a convolutional block, and fully connected layers. Fig 2.4.1 shows the model architecture of ResNet50.

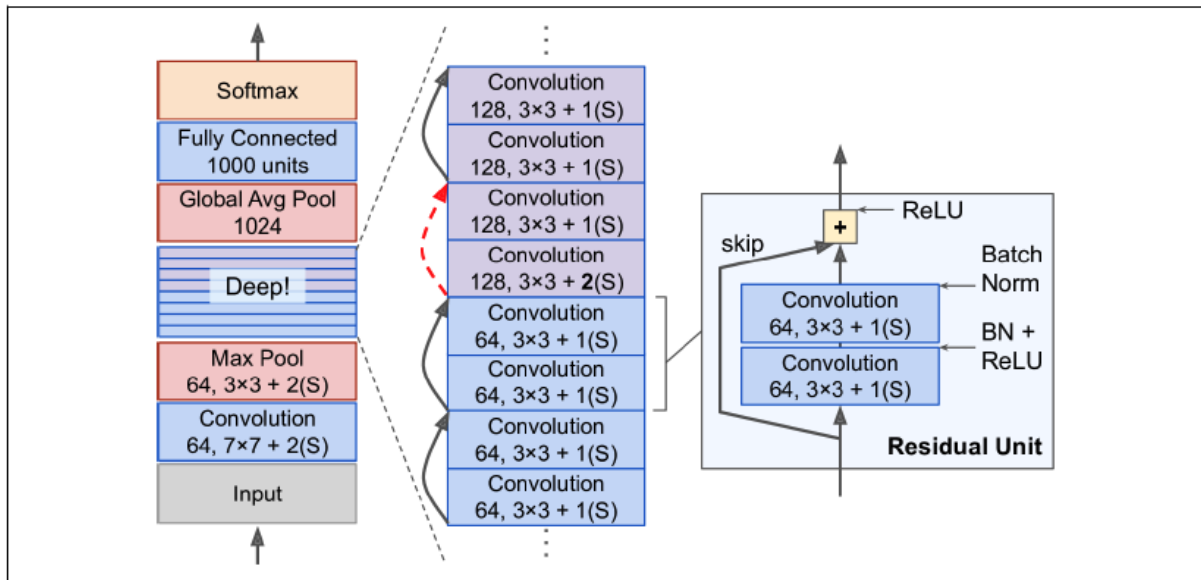


Figure 2.4.1 ResNet50 model architecture [15]

Convolutional layers, as previously discussed, function as feature extractors. The several convolutional layers are followed by batch normalization and ReLU activation. Then max pooling is used to reduce the spatial dimensions of the feature maps. The identity and convolutional block process and transform the extracted features. These are the key elements of ResNet50. The identity block passes the input through a series of convolutional layers and adds the input back to the output, while the convolutional block does the same, but with the addition of convolutional layers that are responsible for reducing the number of filters. ResNet50 is divided into 5 stages, where each stage contains the convolutional and residual blocks. The first stage handles the initial feature extraction, while further stages handle deeper features from the input data. Finally, fully connected layers are responsible for making the final classification. The output is run through a SoftMax activation function that produces the final class probabilities. The feature making ResNet so effective is its ability to skip connections, which can bypass one or more layers. This helps avoid the vanishing gradient problem, allowing

the network to be trained more effectively. ResNet uses a bottleneck architecture. By using a 1x1 convolution to reduce dimensions, a 3x3 convolution to process features, and another 1x1 layer to restore dimensions, instead of stacking three convolutional layers of equal size helps reduce computational cost, while still preserving the network's capacity to learn.

ResNet architectures, including ResNet50, have achieved state-of-the-art performance in many competitions and benchmarks such as ImageNet, and significantly outperformed other architectures like VGG or AlexNet in terms of accuracy. ResNet also proved to be more efficient in terms of computation. [10][11].

2.4. Hierarchical Classification

Hierarchical classification is an approach in machine learning, where labels are organized in a tree-like structure, which allows the classification of data at multiple levels of granularity. This method is particularly advantageous in domains with natural hierarchies, such as biological taxonomy, product categorization, and art classification, where it provides more detailed and contextually accurate predictions [15]. The literature presents several methodologies for hierarchical classification, including top-down, bottom-up, and hybrid approaches. The top-down approach begins classification from the root of the hierarchy, progressively refining the predictions at each subsequent level. This method leverages the hierarchical structure to make broad predictions early on, narrowing down to more specific categories as it moves deeper into the hierarchy. In contrast, the bottom-up approach starts at the leaves of the hierarchy, classifying data into the most specific categories first and then generalizing upwards. Hybrid approaches combine elements of both top-down and bottom-up strategies, aiming to balance the advantages of each, such as accuracy and computational efficiency [16]. The evaluation of hierarchical classifiers often involves unique metrics, such as hierarchical precision and recall (described in Chapter 2.6), which account for the structured nature of the categories [15].

Despite its benefit, hierarchical classification poses significant challenges including computational complexity, especially in deep hierarchies. Overlapping or ambiguous categories can further complicate the training process and reduce accuracy. To combat those difficulties novel architectures have been developed. One example of such architecture was presented in a paper by Yan and Zhand [17]. The paper introduces deep learning models tailored for the

hierarchical classification of text. Although it focuses on text, the techniques and architectures discussed can apply to other domains, including image classification.

There are vast applications of hierarchical classification. For example, in natural language processing (NLP) it is applied to tasks like document categorization, where documents might be first assigned a more general category and then be further categorized into a more specific subtopic [18]. Another field that utilizes hierarchical classification is fashion, where, for example, images are divided into general groups (t-shirt, skirt, dress, shoes) and then each group has subcategories, that specify their type (for example sports shoes, heels, sandals, and boots would be under the category: shoes) [19]. Hierarchical classification is also often used in species classification, where each level of the hierarchy corresponds to a taxonomic rank, such as genus, family, or species [20]. an example of such a hierarchy is presented in Figure 2.4.1. In the medical field, hierarchy is employed in disease diagnosis, where diseases are often grouped into broader categories that can be further divided into specific conditions [21]. These applications highlight the versatility and importance of hierarchical classification across different domains.

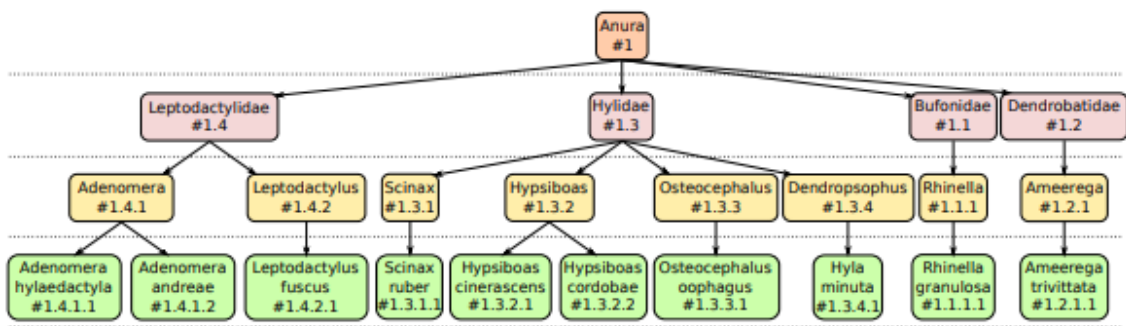


Figure 2.4.1 Example of hierarchy in Colonna, Juan G., João Gama, and Eduardo F. Nakamura including Order, Family, Genus, and Species [20].

2.5. Multilabel Classification

The main difference between multi-label classification and classical classification is that each item of data can have multiple labels and use all of them during the process [22]. For example, in the case of this work, each image will have two labels: artist and style. The most common approach to multilabel classification is Binary relevance (BR). This approach trains a separate binary classifier for each label. Each classifier is then responsible for predicting whether a given instance belongs to its associated label. It does not handle label dependencies assuming they are independent, which might not always be the case. However, thanks to the fact that it is trained independently it can be easily scaled to handle many labels [23].

Another approach to handle multi-label problems is classifier chains, which train a sequence of binary classifiers, where each one predicts whether a specific label should be assigned while using the predictions of the previous classifiers in the chain as additional features. This allows the model to capture label dependencies by incorporating predictions of earlier labels as features for subsequent classifiers, which often improves model performance over BR. However, in this case, the performance can be sensitive to the order of classifiers in the chain, so at times finding the optimal order can be challenging [24].

Powersets (Label Powersets) is another approach to handle multi-label classifications. It trains a single classifier for each unique combination of labels to predict this specific combination. The set of all possible label combinations forms the classes in a multi-class problem. This approach allows for direct modelling of the interactions between all labels by considering all the possible combinations, however, it is computationally expensive and sometimes impractical for problems with many labels [25]. In some cases, where there are more labels, some approaches have been proposed to reduce the demanding computation [26].

In the context of multi-label classification, transfer learning can be effectively utilized by taking advantage of a pre-trained model, such as ResNet, which has already learned extensive feature representations from a large dataset. The pre-trained model's final layers are then fine-tuned to handle the specific multi-label task by adding separate output layers for each label. During training, the model's learned features are refined to capture the nuances of both labels simultaneously, allowing it to generalize well across various categories with minimal additional computational cost. [26][27].

2.6. Understanding and classification of art images in literature

Analysis of images has been done in the literature for decades, with various fields of interest. In recent years, more work has focused on machine understanding of art and its aspects. With the rapidly growing development of machine learning, there was a race to accomplish the best results for various tasks regarding art such as identifying their author, style, or contents. For example, a work by Mao, Cheung, and She [28] focused on learning joint representations of visual arts. In their work, the goal was to generate a representation of visual arts, which would prove to be helpful in other works regarding art analysis. Another work by Lecoutre, Negrevergne, and Yger [29] focused on recognizing art style in paintings using deep learning. For their experiment, they used a popular in the field dataset, called WikiArt [30]. They reached an overall 62% accuracy of style classification on the data. They also highlighted the importance of re-training and concluded that they managed the best results due to the use of residual neural networks. Zhao, Zhou, Qiu, and Juang [31] proposed a different approach, in which, they used a graph CNN and artistic comments to classify the type, school, timeframe, and author of the painting. They implemented natural language processing to accomplish that task. This paper is interesting since it uses a graph based on the association between comments, not the painting itself for the classification. It shows how many various methods are being tried to figure out the best way to identify artworks.

A work by Liu, Yang, Again, and Yuan [32] focuses their classification on portraits only. The goal of their project is to recognize the artistic movement for the portraits using a novel model structure of a CNN. Like most papers in this area of study, they utilized a pre-trained model as a base for their novel approach. For the dataset containing six different art movements, they managed to achieve an average accuracy of 82.6%. Another work focusing on the classification of art style is by Yang [33]. In this paper, the author uses another popular pre-trained model called VGGNET. The accumulated results of the model were compared to classification done by the naked eye gathered from two hundred questionnaires. The comparison shows the VGGNET-based model acquired more accurate results than that of the naked eye by around 4-6% depending on the category of paintings.

When it comes to different models in art classification another paper by Zhao, Zhou, Qiu, and Jiang [9] presented a comparison of their performance. Seven different models were tested to compare their performance on three different datasets. Their abilities were compared based on the classification of genres, styles, and artists. All models had their strengths and

weaknesses, and their performance varied depending on the dataset, type of classification, and used methods for training.

A paper by Hua, Ho, Jangtjiik, and Chen [34] focuses on artis-based classification. In this work, the authors focus on determining the authorship by using Markov random fields with a CNN. The results of this hybrid method proved to be effective and achieved a score of 91.55% accuracy. Another work focusing on artist classification is by de Menezes, Richardson Santiago Teles [35]. A few different CNN architectures were compared (including models like VGG-16, ResNet, and Xception). In this paper, the best result was achieved by Xception, reaching 95% accuracy, while models like VGG or ResNet performed worse, reaching scores of around 60% accuracy. This is interesting when compared to a paper mentioned earlier [9] where ResNet achieved one of the best accuracies out of the compared models. The difference in performance might depend on a lot of aspects such as the chosen dataset, pre-processing methods, hyper-parameter tuning, the nature of the classification (one focused on authorship, while the other included style and genre as well), and other related aspects.

A study by Narvaez, Marcelo Fraile, Ismael Sagredo Olivenza, and Nadia McGowan [36] focused on identifying painting authorship and forgery detection with AI image generation. The work focused on 17th-century Dutch painters, such as Rembrandt, Leveck, or Eeckhout. Three experiments were conducted in this work. One focuses solely on recognizing whether a painting was made by Rembrandt or not. Second, is a model that classifies the authorship of paintings from within the artists included in the work. Lastly, the model attempted to classify paintings generated by AI using the categories from the previous experiment. The accuracy of the best-performing instances of the experiments reaches over 90%.

In recent times more works are showing up regarding negative bias and perception of AI by humans. One such work by Chiarella, Salvatore G., et al. [37] investigates a different aspect of AI and automatic authorship assignment. Effects of authorship assignment done by AI are studied in the light of negative bias towards artificial intelligence and their mutual interaction. The paper posed the question of whether human-declared vs. AI-declared authorship of specifically abstract paintings influences aesthetic appreciation towards AI and investigated the negative bias towards AI-declared abstract work.

2.7. Evaluation Metrics

This chapter delves into evaluation metrics and methods to ensure their reliability. One of the key metrics in model evaluation is accuracy, which indicates the percentage of correct predictions. Accuracy is used to evaluate the generalization of the trained classifier. It refers to correctly predicted data from a training set. Another related metric is training accuracy which indicates how well the model performs on the training dataset during the training process, usually checked after each epoch [38][39].

Another evaluation metric often used in classification is precision, which measures the accuracy of positive predictions. Precision is defined as the ratio of correctly predicted positive observations to the total predicted positives.

$$Precision = \frac{True\ Positives}{True\ Positives + False\ Positives}$$

High precision indicates a small number of false positives.

Recall, also called Sensitivity or True Positive Rate is another evaluation metric. Recall is the ratio of correctly predicted positive observations to all observations in the actual class [40].

$$Recall = \frac{True\ Positives}{True\ Positives + False\ Negatives}$$

This metric measures the ability of the model to capture relevant instances. High recall indicates a small number of false negatives [40].

F1 Score as an evaluation metric is the harmonic mean of precision and recall. It provides a single metric that balances both precision and recall. The F1 Score is especially useful when class distribution is imbalanced.

$$F1\ Score = 2 * \frac{Precision * Recall}{Precision + Recall}$$

A high F1 score indicated a good balance between precision and recall [41].

In the case of the hierarchical classification, there needs to be some adjustments made to the calculations of Precision, Recall, and F1 score so that they accurately represent model performance. For multi-label classification, particularly one using BR, the metrics could be calculated separately for all labels, according to the provided formulas. For an overall calculation precision, recall and F1 score could be derived by averaging the metrics across both labels. However, when considering an overall metric, a hierarchical precision and hierarchical recall to account for situations where partial correctness (correct style, incorrect artist or incorrect style, correct artist) is acknowledged should be considered.

For precision calculation, both labels should be considered to reflect the performance of the classification better. The formula is the same, but the calculations of True positives and false/true negatives are done as follows:

$$TP_{hier} = \sum_{i=1}^n (Full Match + Partial Match)$$

Here, a full match reflects the situation where both labels are identified correctly, while a partial match is a situation where only one of two labels is correct. In this case full match would count as 1, and a partial match as 0.5. This formula applies to a situation where there are two labels, but it could easily be extended to take into account more labels.

False positives are calculated as follows:

$$FP_{hier} = \sum_{i=1}^n (\prod [Predicted Label1_i = 1 \wedge True Label1_i = 0] \\ + \prod [Predicted Label2_i = 1 \wedge True Label2_i = 0])$$

$$\prod [condition] = \begin{cases} 0.5 & \text{if the condition is true} \\ 0 & \text{if the condition is false} \end{cases}$$

This sum counts cases where either or both labels were incorrectly predicted, considering each incorrect prediction as a false positive. False negatives occur when the model fails to predict a correct label that is actually present in the true labels.

False negatives are computed in a similar manner but focus on what the model failed to predict but should have, and are calculated as follows:

$$FN_{hier} = \sum_{i=1}^n (\prod [True Label1_i = 1 \wedge Predicted Label1_i = 0] \\ + \prod [True Label2_i = 1 \wedge Predicted Label2_i = 0])$$

This sum counts cases where the model failed to predict the correct labels, considering each missed label as a false negative. This formulation mirrors FP but from the perspective of what was missed in the ground truth, emphasizing that FN is about failing to predict something that should have been predicted.

Finally, precision, recall, and F1 Score are calculated as follows:

$$Hierarchical Precision = \frac{TP_{hier}}{TP_{hier} + FP_{hier}}$$

$$Hierarchical Recall = \frac{TP_{hier}}{TP_{hier} + FN_{hier}}$$

$$Hierarchical F1 = 2 * \frac{Hierarchical Precision * Hierarchical Recall}{Hierarchical Precision + Hierarchical Recall}$$

This method of calculating hierarchical precision and recall focuses on the hierarchical relationship by giving partial credit for correct predictions at different levels, without the need to introduce an additional weighting factor. It also distinguishes between precision (focusing on false positives) and recall (focusing on false negatives), which is important for understanding how well the model balances between over-predicting and missing correct labels. This approach helps take into account the hierarchical character of the model, by including partial matches in the calculations. [15][42]

Some techniques were also utilized to improve the reliability of evaluation metrics. One such technique is validation, which evaluates the model on a separate dataset (validation set) that was not used during training. This is done to help assess the model's performance, as well as its generalization ability. The idea is to make sure the model is not just memorizing the train

data but is able to make accurate predictions on previously unseen data. Validation is typically performed after each epoch and offers a deeper insight into the model's performance [43].

Another method is cross-validation, which is often used as a data resampling method for model selection and evaluation. It often helps with overfitting and helps estimate the generalization of the model. Cross-validation is done by splitting the dataset into subsets called folds. The model is then trained on several folds while being validated on the remaining fold. This process is repeated so that each fold serves as the validation set once. At the end results from each fold are averaged to provide an overall performance metric. This helps ensure the model performs well on previously unseen data [44].

Another technique to avoid overfitting is early stopping. This method simply stops the training before convergence can happen. The early stopping method monitors model performance on the validation set and stops the process when performance no longer improves. For example, the training would be halted when validation loss stops decreasing or validation accuracy stops increasing. This method ensured that the model generalized better on unseen data rather than continuing to learn patterns specific to the data on which it was trained [45].

3. Implementation

The following chapter will focus on the implementation of the project. First, the dataset and its preparation are discussed, then extraction of fragments for the training process, data augmentation, and finally, all three types of models are described.

The basis of the project is a code created in the Python programming language. Multiple Python libraries were used to achieve the expected results including os, random, albumentations, cv2, h5py, matplotlib, numpy, pandas, PIL, torch, sklearn, seaborn, and optuna. A pre-trained ResNet50 model has been utilized as well, for transfer learning.

3.1. Dataset

To ensure a good distribution of labels in the dataset a few available sources of paintings were used: a Kaggle dataset [46], Wikiart paintings [47], Wikipedia paintings [48], and museums' online archives [49][50]. Image 3.1.1 shows a few examples of paintings from the dataset. All works used in this project are utilized under fair use or appropriate licensing, allowing their inclusion for this purpose.

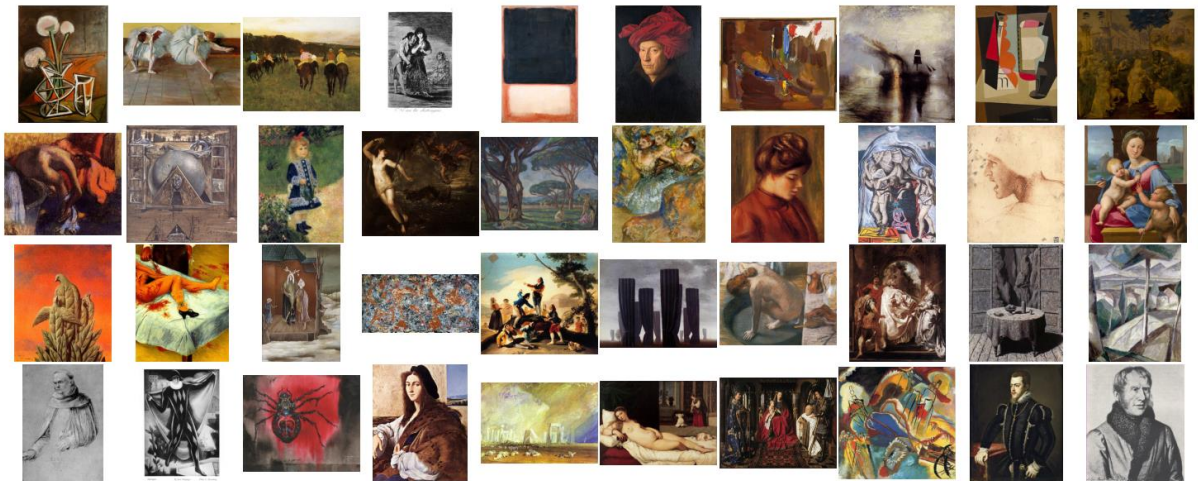


Fig 3.1.1. Examples of images from the dataset.

The Kaggle dataset, called “Best Artworks of all Time”, consists of 49 artists and 44 styles. Some artists, however, had to be excluded since their paintings belonged to multiple styles, some were filtered to contain paintings of only one style. Some styles were excluded

completely due to resource limitations. Wikipedia paintings as well as online museum archives provided additional artists and styles to improve the balance in the dataset and expand it for the purpose of this work. One of the models created in this project requires that in the dataset each artist has paintings in only one style thus the artists' works were limited to those within one specific style, usually one for which the artist is most known. The final version of the collected data contains 7 styles and 43 artists. Figures 3.1.2 and 3.1.3 show their distributions in the dataset, while Figure 3.1.4 sums up the number of artists per style.

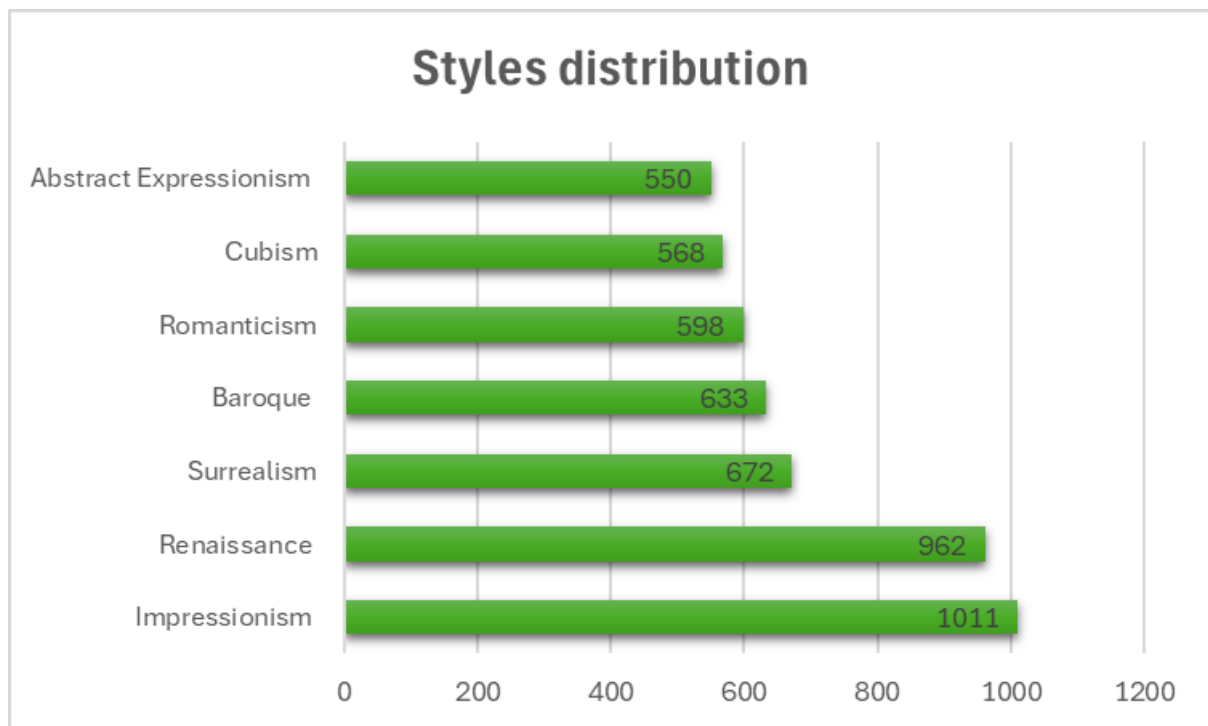


Figure 3.1.2. Distribution of styles in the dataset.

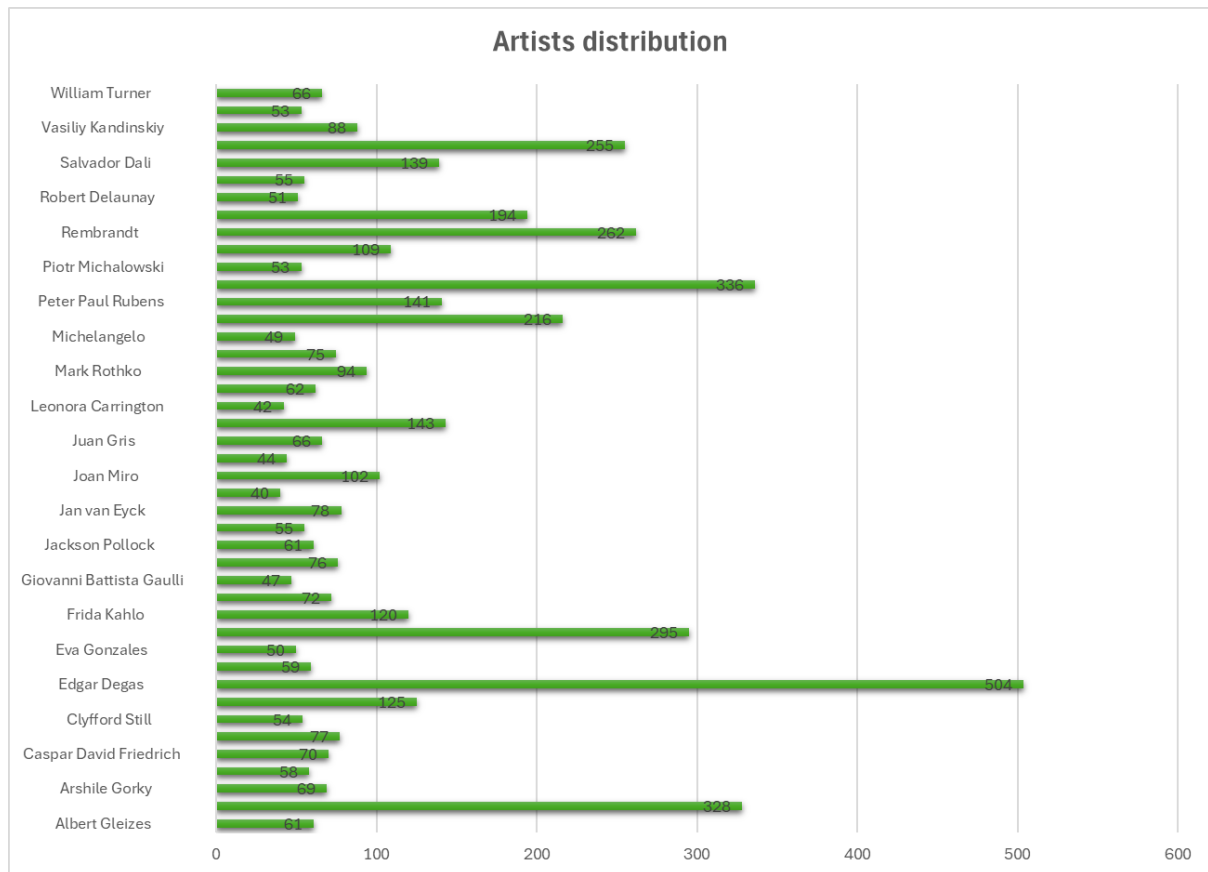


Figure 3.1.3. Distribution of artists in the dataset.

Style	Number of artists
Abstract Expressionism	8
Baroque	5
Cubism	7
Impressionism	5
Renaissance	6
Romanticism	6
Surrealism	6

Figure 3.1.4. Number of unique artists per style.

While creating the dataset the idea was to ensure a well-distributed set of styles, while not overwhelming the dataset with too many artists per style and keep their number quite equal

as well. Each style is represented by 5-8 artists. While with style it was a bit easier to keep the balance, it was more difficult for artists, since for a lot of them only a few paintings are available online. The difficulty was also posed by the varying resolution of paintings, which later influenced the number of fragments that could be extracted. Thus, the distribution of paintings per artist fluctuates more than the distribution of paintings per style. Figure 3.1.5 shows the structure of the dataset.

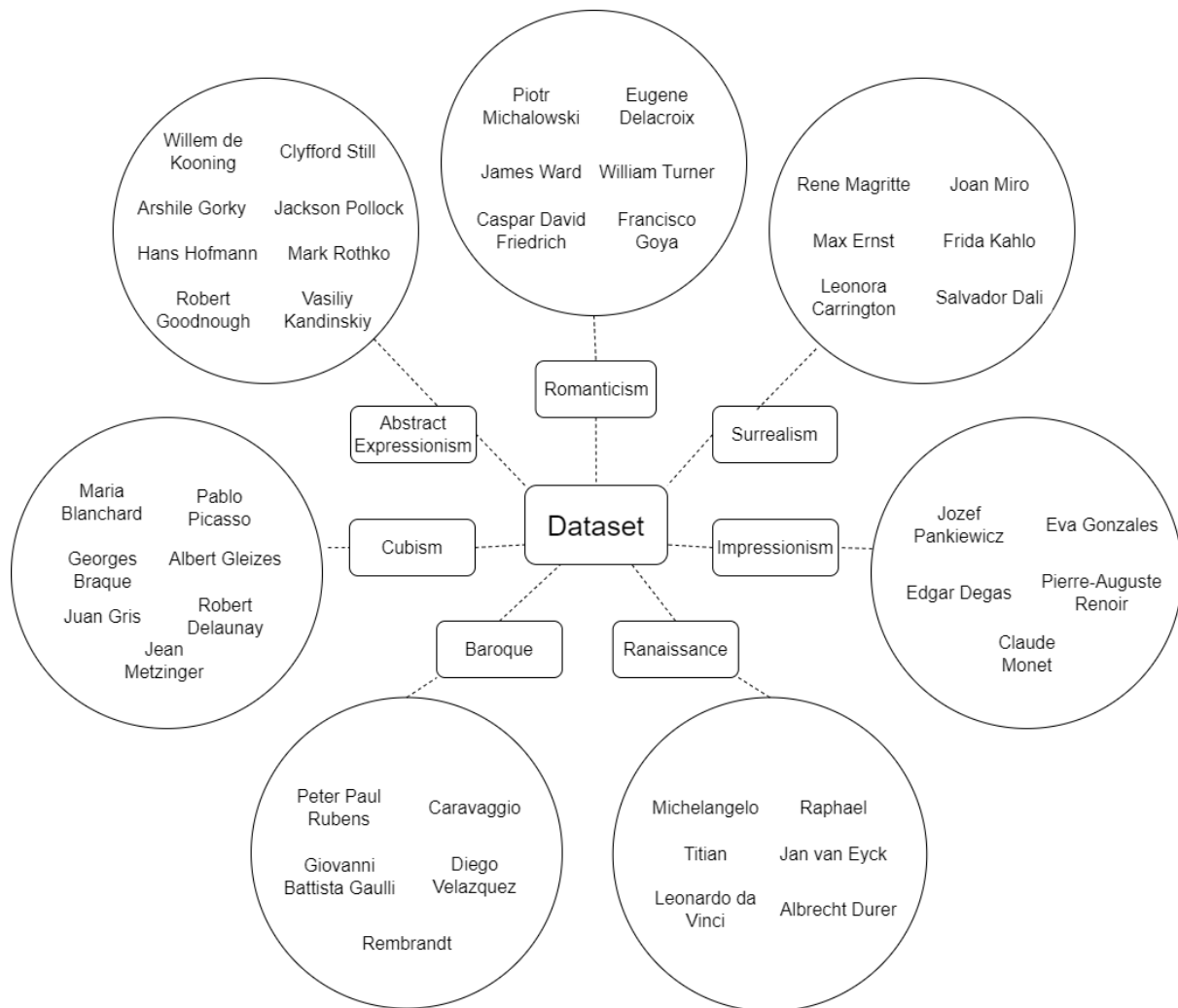


Figure 3.1.5. Diagram of the structure of styles and artists in the dataset.

3.2. Fragment extraction

Since the idea of this project was to utilize fragments of paintings for the training process there is a need to extract said fragments first. The idea of the extraction was to try and grab as many fragments as possible from each painting without having too much overlap. That might not always be possible, since the available paintings are often in varying image quality, some images being even smaller than the desired fragment size.

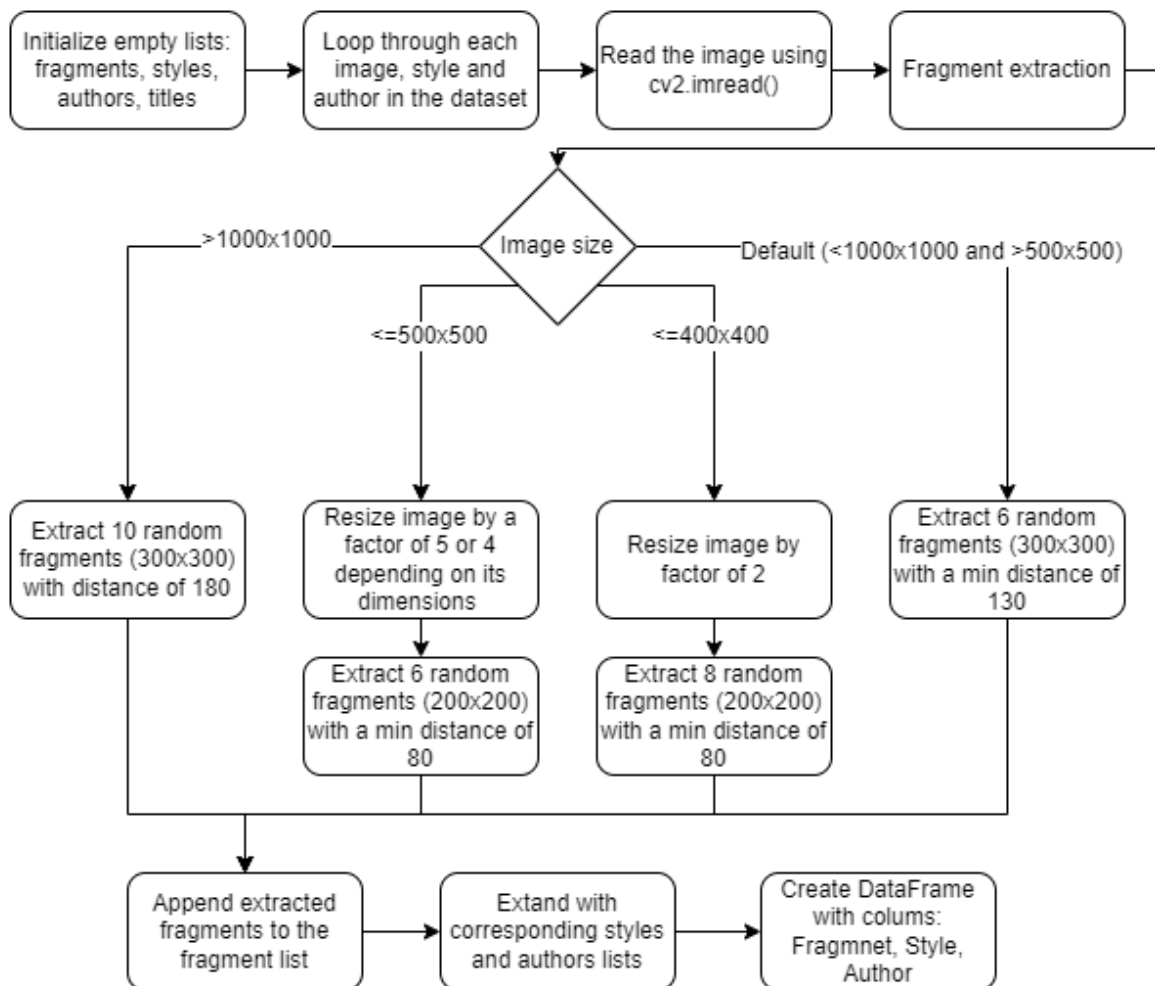


Figure 3.2.1. Flowchart of the fragment extraction process.

Fig 3.2.1 presents the flowchart of the fragment extraction process. The code designed for this part of the work starts from list initialization. The next part reads the images along with their metadata that contains all the necessary information like the author and style. Then, multiple random fragments of specified size are extracted for each image. The number of fragments depends on the original image size. Smaller images are first resized to be larger, and

the number of fragments extracted is less than for larger images where the resizing is not required. The extracting is done using an additional helper function that makes sure the fragments are far enough from each other based on a minimum distance criterion. That is done to ensure fragments are sufficiently spaced apart to avoid large overlaps. Finally, the fragments are stored in a dataset along with their authors' and styles' names. Figure 3.2.2 shows examples of the extracted fragments.

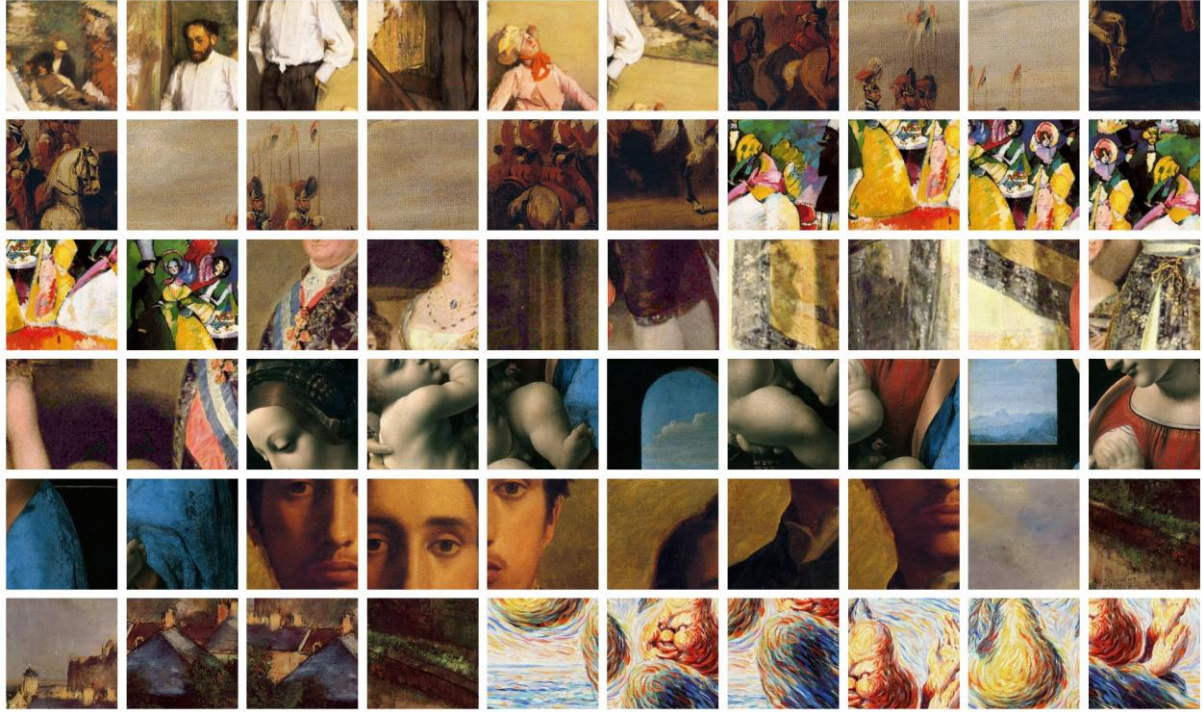


Figure 3.2.2. Examples of extracted fragments.

3.3. Data augmentation

For pre-processing of the training data methods that are most common in data augmentation were used [9][51]. For this purpose, the Python library called ‘alumentations’ was utilized [52]. Very few functions were used for the augmentation since this work does not focus on the analysis of pre-processing and its influence on the accuracy of classification. Four usually used methods were applied to the dataset from the alumentations library: horizontal flip, vertical flip, random resized crop, and color jitter. Horizontal and vertical flips transform the input image by flipping it horizontally around the y-axis and vertically around the x-axis, respectively. Random resized crop takes a random part of the input and rescales it to a set size.

Finally, color jitter randomly changes the brightness, contrast, and saturation of the input image. Figure 3.3.1 shows examples of fragments after augmentation.



Figure 3.3.1. Example of images after augmentation.

During pre-processing, the images were also normalized to improve performance, stabilize the training process, and prevent overfitting. Normalizing entails scaling pixel values to a specific range. In this project, the normalization is done using PyTorch’s transform function ‘Normalize’ [53], which adjusts the pixel values to have a specified mean and standard deviation.

Finally, the data was converted to tensors, which means transforming images into a tensor format. Tensor is a multi-dimensional array of numerical values. This was done using PyTorch’s torch-vision function called ‘ToTensor’ [54]. This conversion is necessary to provide compatibility with the created models and ensure a standardized format of the input data.

To give the models a possibility of later comparison, data for all the models was augmented in the same way, as well as the same dataset was used. This is done to not leave the final differences in the results up to the differences in data augmentation but to the different models and architectures used.

3.4. Recognizing painters and on this basis inferring style

The method recognizing first author then style (AtS) described in this chapter is the most straightforward out of all the methods used in this project. This approach involves first classifying the author of the painting, and once the painter is identified, the style is assigned using an artist-style mapping, based on the dataset’s structure. This method requires only one

classifier that will recognize the author of the painting. In this approach, the accuracy of the style classification is directly correlated to the artist classification.

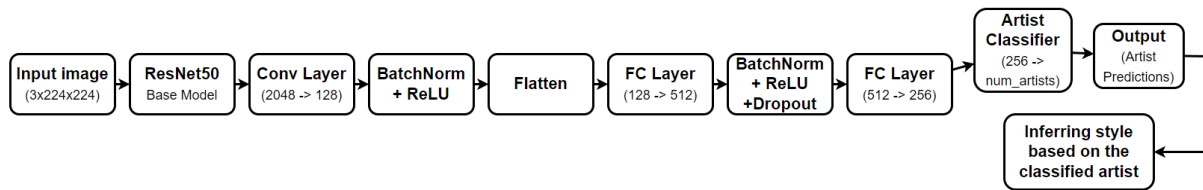


Figure 3.4.1 General structure of the model recognizes artists first and then infers style.

Figure 3.4.1 shows the architecture of the model used for the AtS model. The model incorporates a pre-trained ResNet50 with custom layers to refine and adapt the feature for the specific task of artist and style classification. The convolutional layer (ConvLayer) reduces the number of feature channels, making the model focus on the most relevant features, while also making the following layers more efficient. Batch normalization (BatchNorm) normalizes the activation, which stabilizes and accelerates the training process. Rectified Linear Unit (ReLU) is an activation function used to apply non-linearity, which allows the network to learn more complex patterns and representations. Another layer (Flatten) is a step that prepares the data input into the fully connected layer by flattening the multi-dimensional vector into one dimension. FC Layer is a fully connected layer, which is followed by batch normalization, ReLU, and Dropout. A dropout layer randomly sets a fraction of the input units to zero during training, helping to prevent overfitting by encouraging the network to learn more robust features. Another fully connected layer is added after. Finally, a classifier for artists is introduced. The classifier maps the features to the number of artists, enabling the network to predict which artist is most likely to be correct for the input image. Finally, the model produces the output author and then based on that assigns the style associated with the recognized artist. To avoid overfitting validation data was used to verify the performance of the model after each layer.

It is important to mention that, while this model might give very accurate predictions, it will not be at all reliable in cases, where artists have paintings in more than one style. Thus, in the future, further work could be done to expand this approach and include a second classifier that would handle recognizing which style from ones the artist creates in would categorize the painting, similarly to the approach presented in chapter 3.6.

3.5. Recognizing authors and styles simultaneously

The model recognizing both artists and styles simultaneously (AaS) uses a multi-label approach, where two labels are being classified simultaneously. Such models are often used in the field of deep learning, however, no works that use it could be found in the area of art. Thus, this model is created to observe whether that kind of approach could provide better results than other models tested in this project.

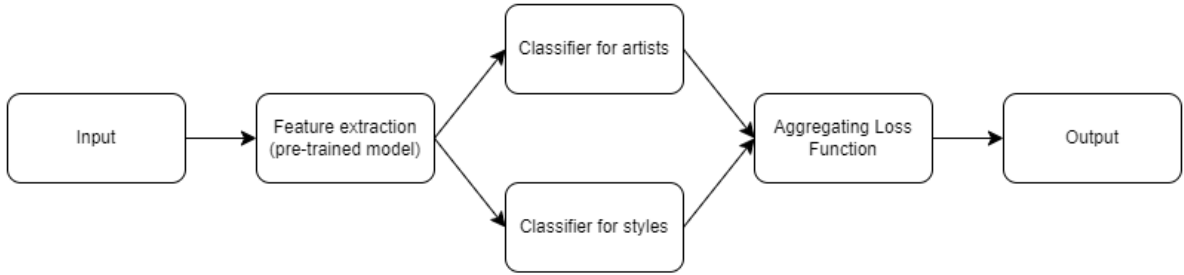


Figure 3.5.1. Diagram structure of multilabel classification.

In Figure 3.5.1 general structure of the multilabel approach, used in this project is presented. It focuses on the learning process and how both labels are handled during training. Used pre-trained model serves as an encoder for feature extraction. It processes the input image to extract high-level features. Subsequently, two separate fully connected layers handle the multilabel classification, predicting the artist and style labels independently. For each label, an ensemble of binary classifiers is employed based on the Binary Relevance (BR) method. This ensemble approach involves training multiple binary classifiers for each label and aggregating their predictions to improve robustness and performance. Finally, an aggregating loss function is used to combine the losses from both classification heads, which gives a unified measure of model performance.

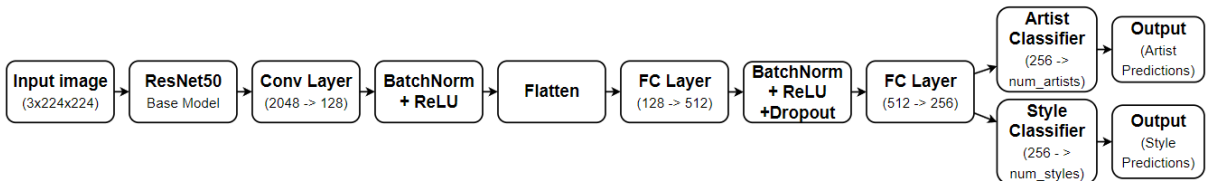


Figure 3.5.2. General structure of the model recognizes both artists and styles simultaneously.

Figure 3.5.2 shows the architecture of the model used for this dual classification. The structure of the model is analogical to the AtS model. The main difference is in the number of classifiers and how they are handled. Classification for artists and styles is done separately and the model produces two outputs: one for author prediction and one for style.

3.6. Two-stage classifier that first recognizes the style and then the author

This chapter introduces the final model created for this project in which multiple classifiers are created: one for inferring style and seven for artists within each style (StA).

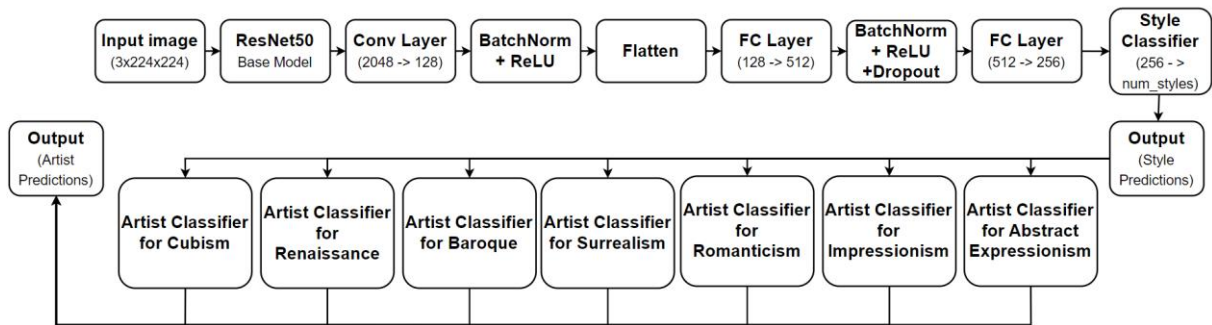


Figure 3.6.1. General structure of the model recognizes both artists and styles simultaneously.

The architecture of this model is similar to the two previous approaches, differentiated by only the classifiers and their setup. StA first classifies the style and then creates separate classifiers for the artists within each style. Seven classifiers are created, one for each style in the dataset.

4. Results

This chapter aims to present and analyze the results obtained from each model, providing a detailed evaluation of their performance across key metrics. The outcomes will be compared to identify patterns, strengths, and weaknesses in each model's approach. The goal is to determine which model proves to be the most effective and reliable for the given task, ensuring a thorough and objective assessment.

4.1. AtS Model

This model reached an accuracy of 66,74% for artists and 78,84% for styles. Figure 3.4.2 shows the training and validation loss and accuracy for the artist classification over ten epochs. The training loss consistently decreases, which suggests that the model is effectively learning from the training data. However, the validation loss initially decreases but begins to plateau and even increases slightly around epoch 6, indicating potential overfitting. This suggests the model is learning patterns in the training data that do not generalize well to the validation data. Training accuracy improves steadily, but validation accuracy reveals that while the model is becoming better at classifying the training data, it is not improving its performance on the unseen validation data. The gap between training and validation curves in both plots points to a problem of overfitting, where the model becomes too tailored to the training data.

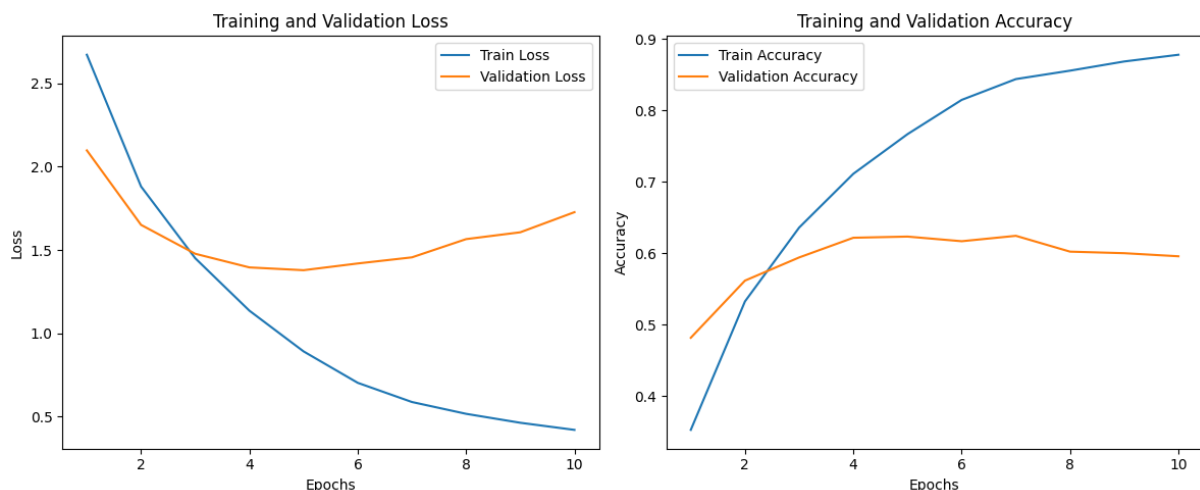


Figure 3.4.2. Training and validation loss for artist classification for the AtS model.

In Fig 3.4.3 which presents a confusion matrix for artist classification, it can be seen that most labels were correctly identified, with a few outliers that are spread over multiple labels. Most common misclassifications occur for Paul Rubens (Baroque), who is confused with Diego Velazques (Baroque), Eugene Delacroix (Romanticism), and Francisco Goya (Romanticism). Hans Goffman (Abstract Expressionism) was confused by the model with Clyfford Still (Abstract Expressionism). These misclassifications suggest that the model has learned some stylistic patterns, as many errors occur between artists within the same style, such as Baroque or Romanticism. This is a positive sign, as it shows that the model is capturing stylistic features shared by artists within the same style, which is preferable to confusing artists across entirely different styles, where the visual characteristics are likely to be more distinctly different.

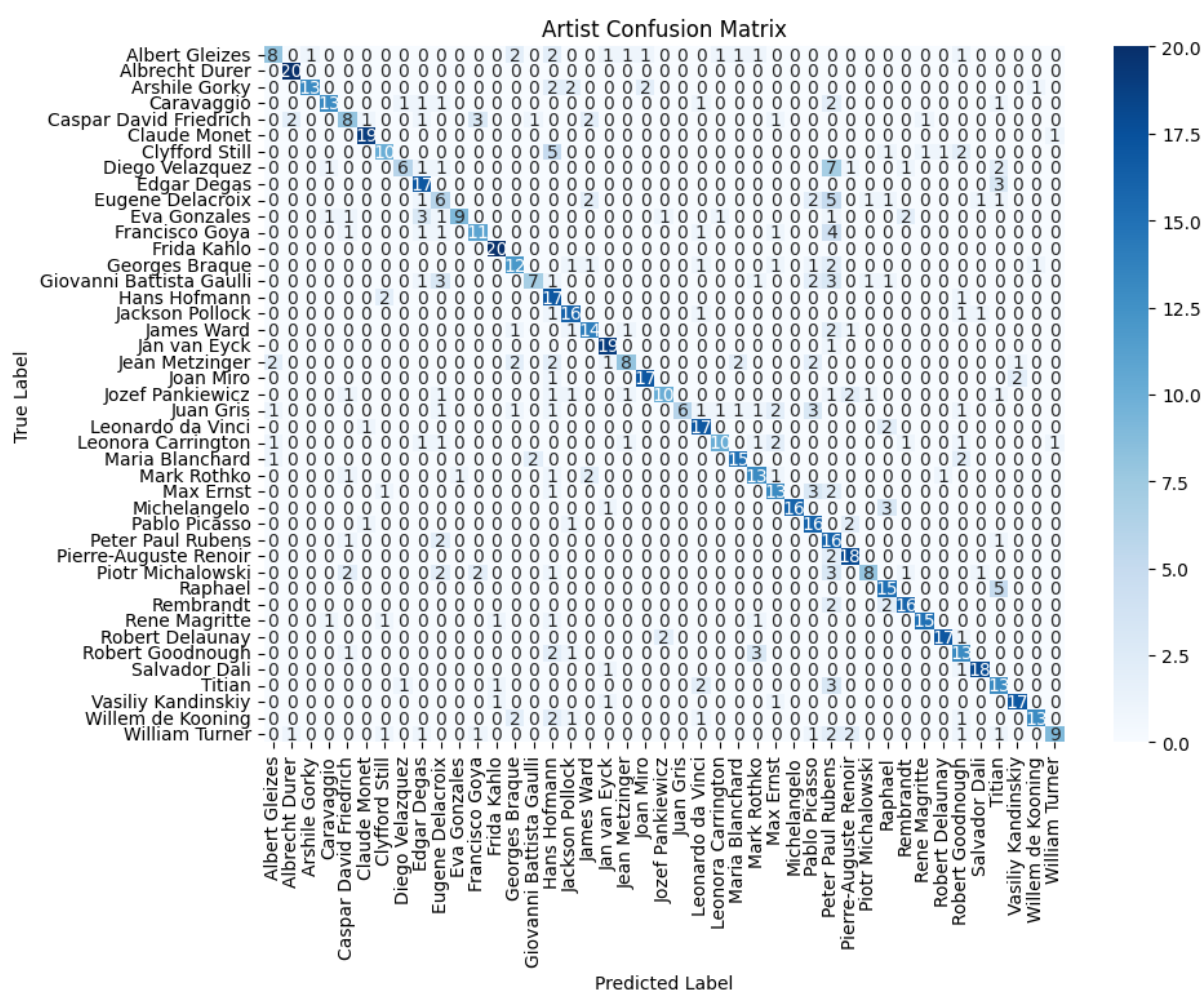


Figure 3.4.3. Confusion matrix for artists for the AtS model.

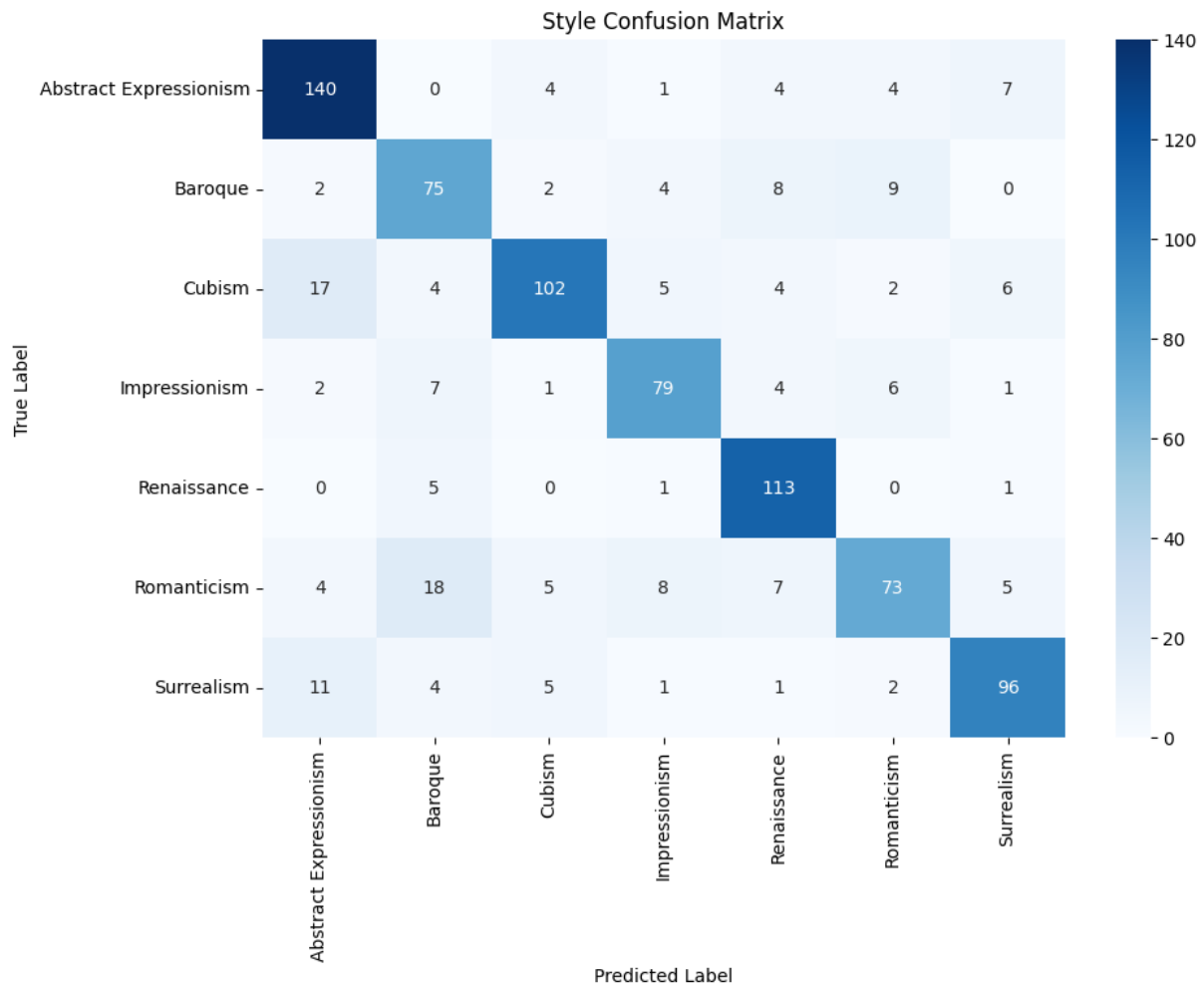


Figure 3.4.4. Confusion matrix for styles for the AtS model.

The confusion matrix for styles (Fig 3.4.4) shows that the model confused Cubism, Abstract Expressionism, and Surrealism the most. Other more common misclassifications are between Baroque and Romanticism. There aren't many incorrect classifications between styles that don't have certain similar characteristics, which again suggests that the model was able to capture general stylistic features for the styles. The few misclassifications between styles that bear little to no similarities might result from the data used for the training, which often contains fragments with very little information that could help the model correctly identify the right style/artist.

4.2. AaS model

For the AaS model, the final results reached an accuracy of 60,05% for artists and 80,65% for styles. Due to the limited number of images per artist, it was expected that authorship classification would perform worse.

Figure 3.5.3 presents the plot of training loss and accuracy curves for artist and style classification over ten epochs. For both, the loss decreases over time, which indicates the model is learning during the process. However, both accuracy and loss seem to plateau early, suggesting limited improvement as training progresses. The curves in the plot might imply potential overfitting or difficulty in optimizing the model for both artist and style classification. The model might be optimizing the wrong aspects of the problem, learning patterns that don't generalize well to accuracy improvements. This could indicate the need for better regularization tuning or more complex model architecture to better capture the nuances of the classifications. Figures 3.5.4 and 3.5.5 present confusion matrixes for both classifications, which will allow to take a better look at the results of the classifications.

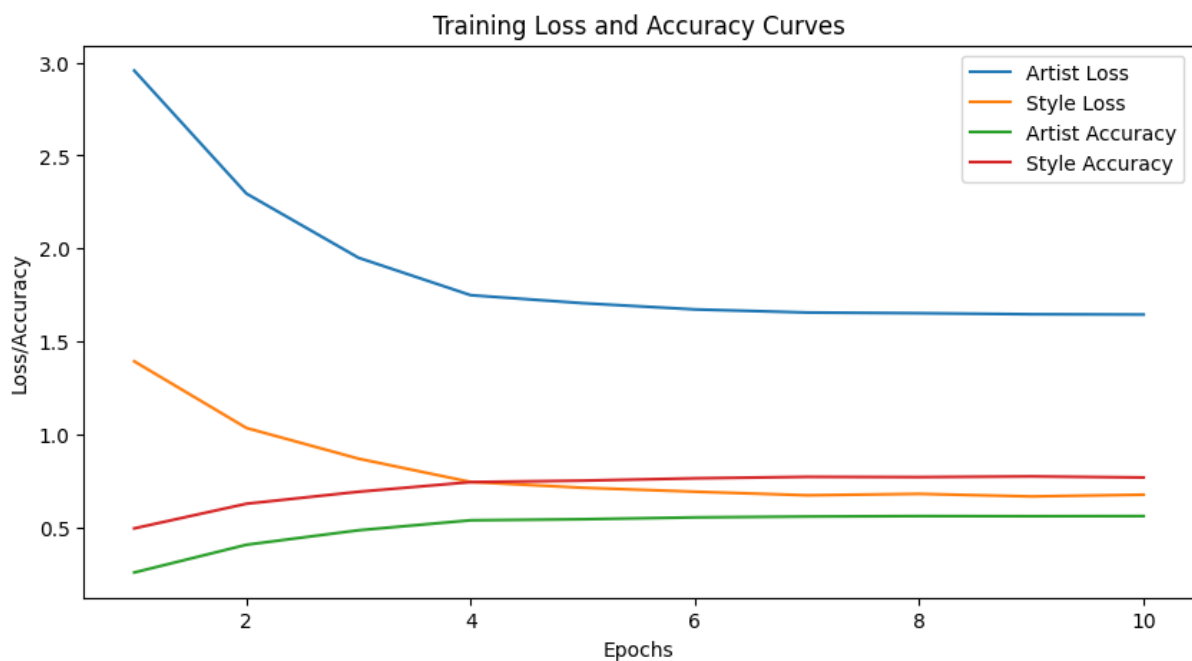


Figure 3.5.3. Training loss and accuracy for the AaS model.

In the artist classification, as seen in the confusion matrix (Fig 3.5.4), Francisco Goya was confused with William Turner, Piotr Michałowski, Eugene Delacroix, James Ward, and Caspar Dabid Friedrich who all represent the same stylistic movement as Goya, as well as Eva

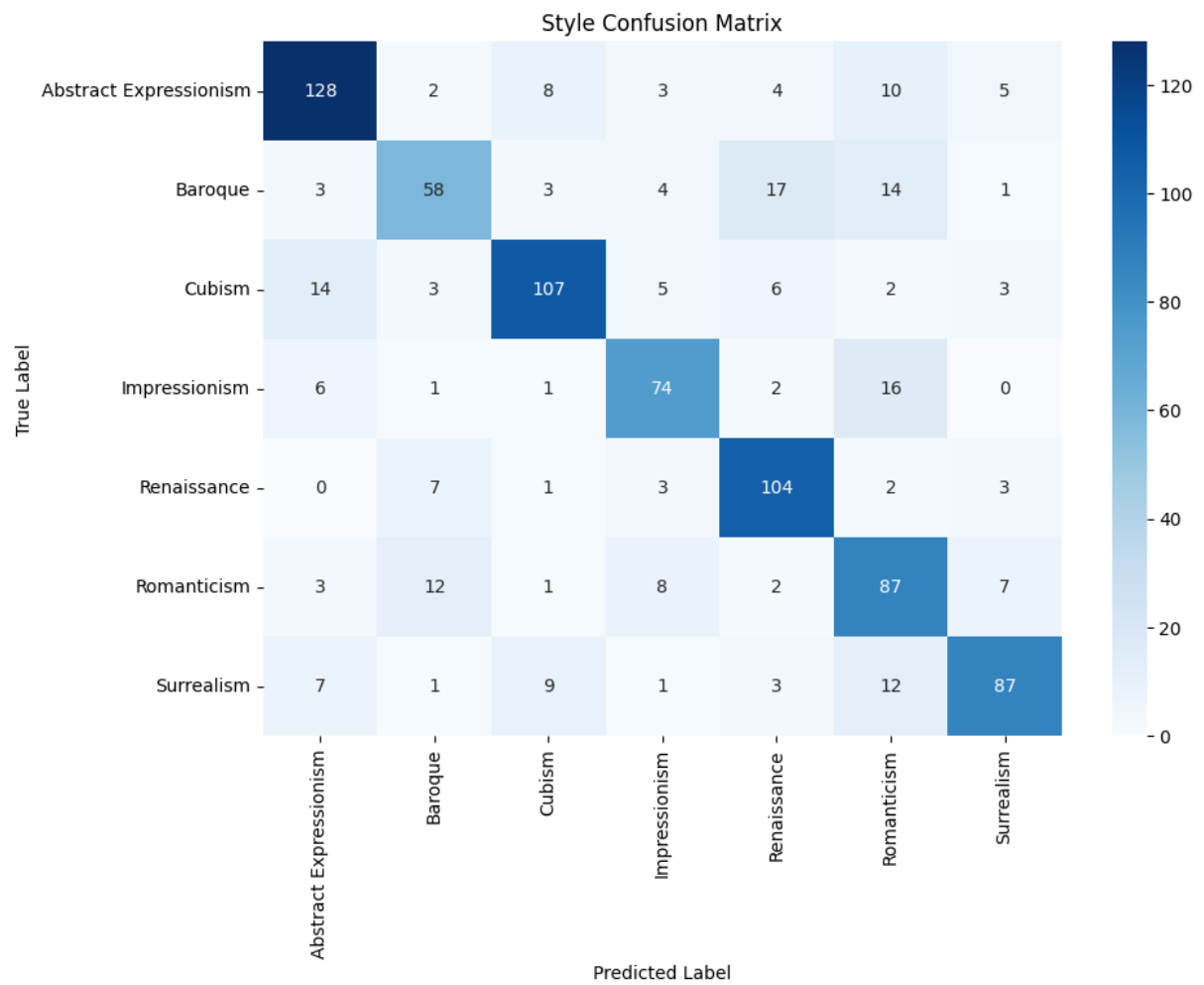


Figure 3.5.5. Confusion matrix for style classification.

4.3. StA Model.

For style training the model reached an accuracy of 77,93%.

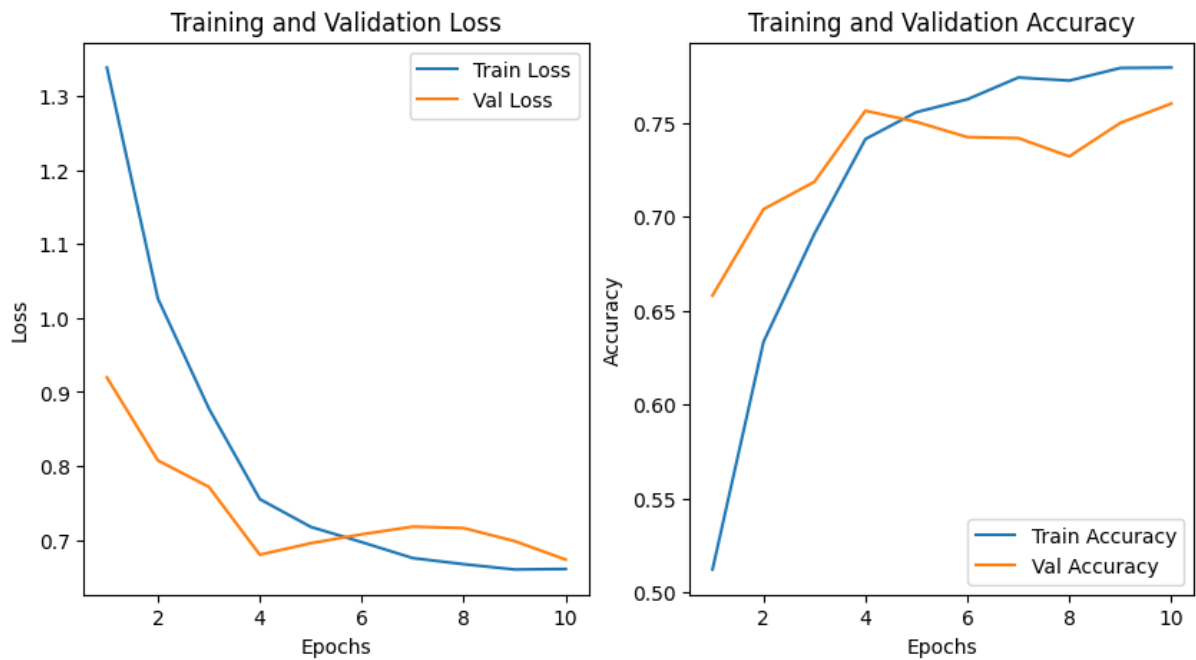


Figure 3.6.2. Training and validation loss and accuracy for style classification in the StA model.

Figure 3.6.2 presents plots for the training and validation loss and accuracy for the style classification over ten epochs. The training loss steadily decreases, indicating that the model is effectively learning from the training data. The validation loss shows a similar decreasing trend, with some fluctuations around epochs 4 to 8, but it ultimately reduces, suggesting decent generalization to the validation data. For accuracy, both training and validation accuracy increase significantly in the first few epochs. The validation accuracy plateaus around 0.75 after epoch 5, while the training accuracy continues to rise slightly, reaching around 0.8. The relatively small gap between the two accuracies suggests that overfitting is less severe compared to the previously analyzed examples, and the model is generalizing reasonably well on the validation set. Overall, the model shows good performance in style classification, with minimal overfitting and a consistent improvement in both loss and accuracy for both training and validation sets.

Artists were trained separately for each style. For Abstract Expressionism, the accuracy reaches 42,34%. From the training and accuracy curves (Fig 3.6.3), it can be seen that the model could benefit from longer training. That was not possible due to time limitations. The loss is decreasing, while the accuracy increases. At this stage, the model doesn't show signs of overfitting.

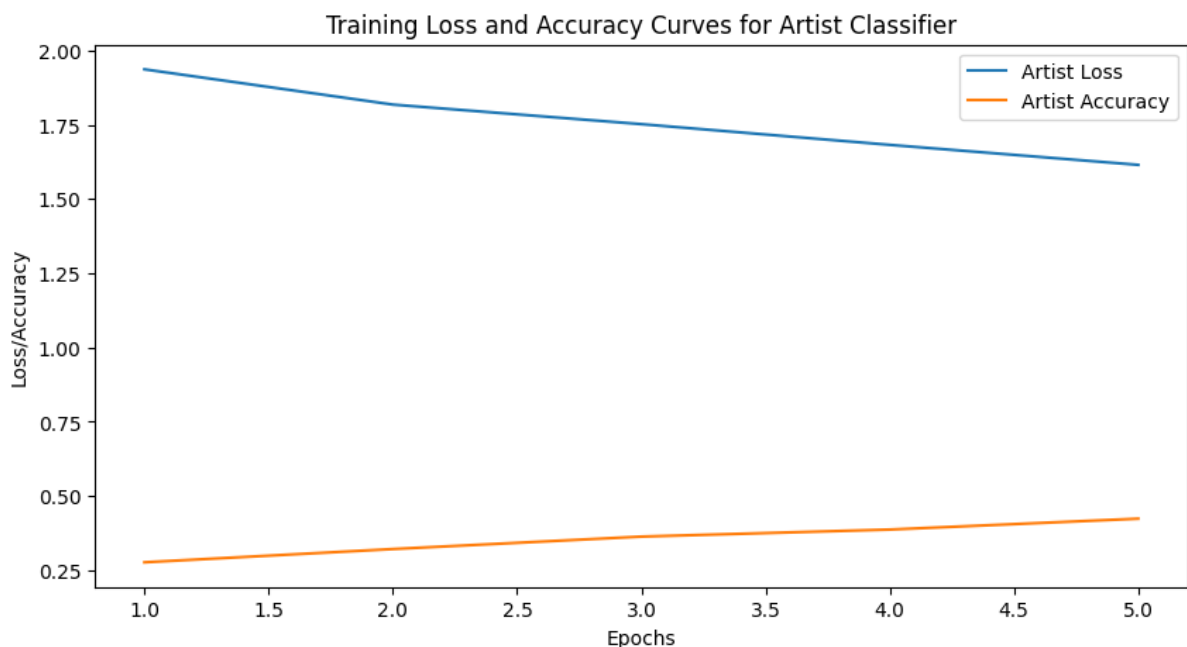


Figure 3.6.3. Training loss and accuracy for artists within the Abstract Expressionism style.

For all the artists in the remaining styles, the loss and accuracy curves (Fig 3.6.4) take a similar form. It seems like all of them could benefit from further training, which was not possible due to time limitations. At their current stage, the models don't show signs of overfitting. In all of them, the accuracy is on the incline while loss decreases.

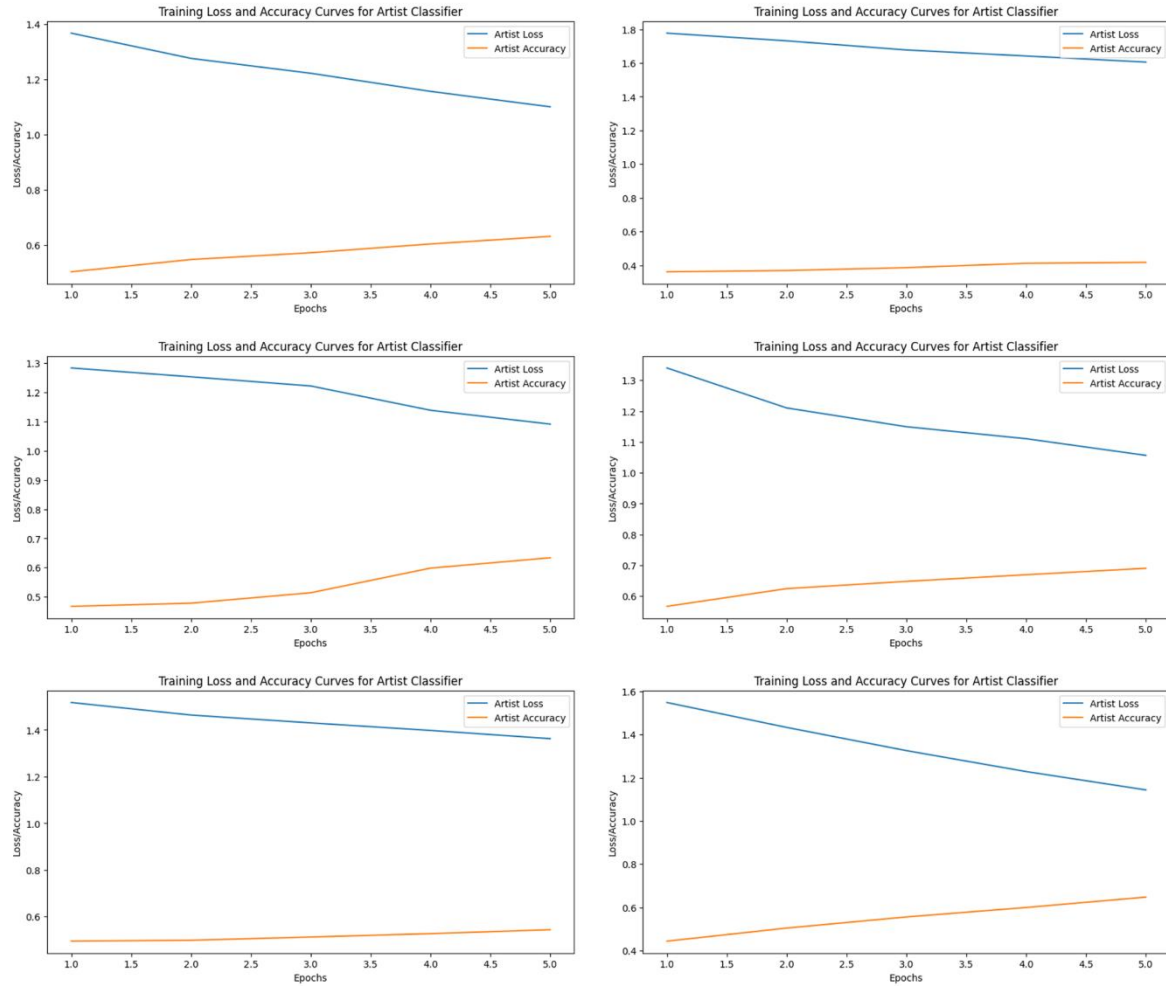


Figure 3.6.4. Training loss and accuracy for artists within the Baroque (top left), Cubism (top right), Impressionism (middle left), Renaissance (middle right), Romanticism (bottom left), and Surrealism (bottom right).

The confusion matrix for styles (Fig 3.6.5) shows misclassifications similar to those in the previous models, where styles with similar qualities are often confused with each other.

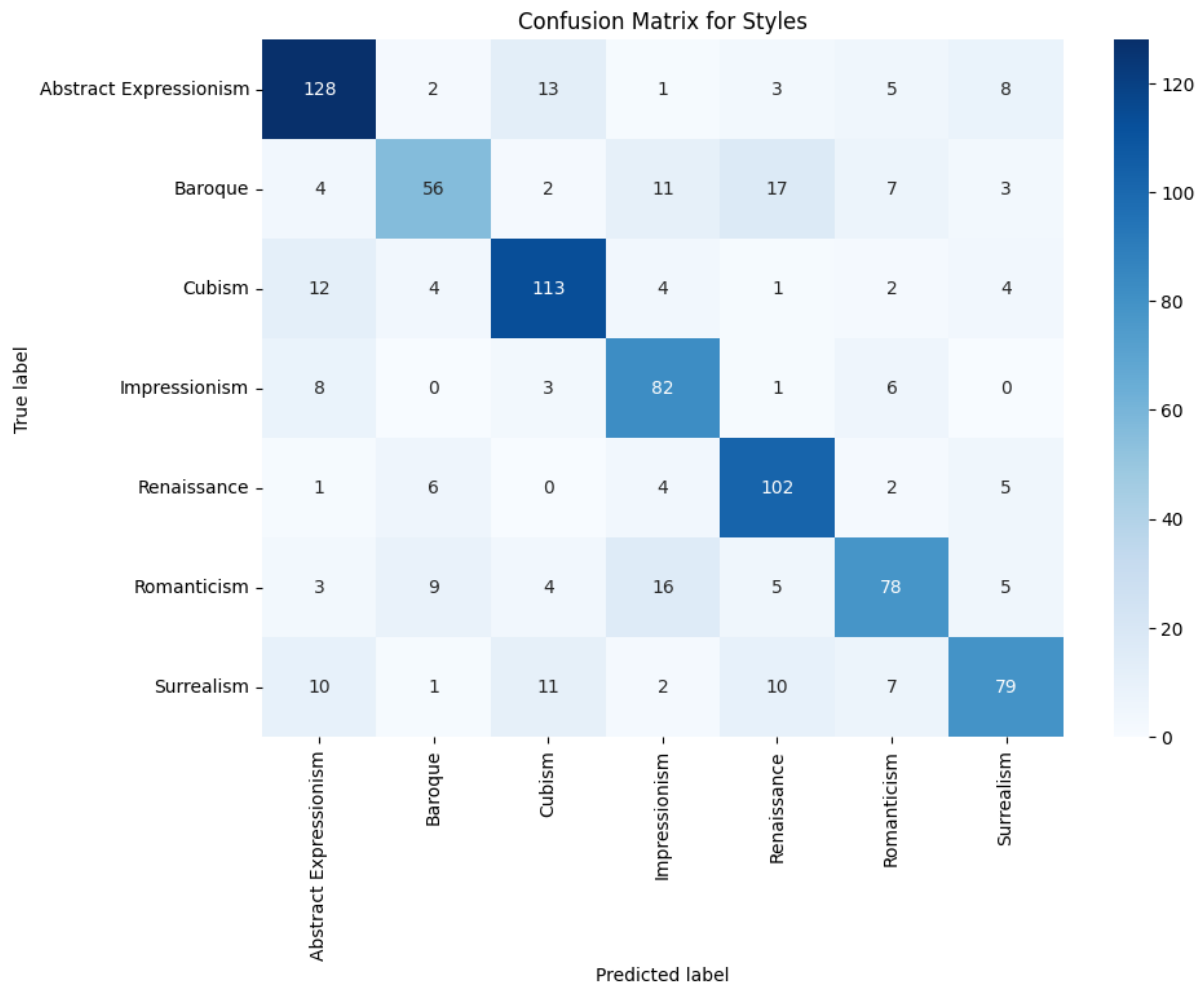


Figure 3.6.5. Confusion matrix for styles for the StA model.

While analyzing the confusion matrices for artists (Fig 3.6.6, Fig 3.6.7 and Fig 3.6.8) there is a need to consider that some of the results might be influenced by the style classification. Since the data fed to artists models is based on the previously done style classifications, there is a risk of some artists being put in the wrong category, thus making it impossible for the model to classify them correctly.

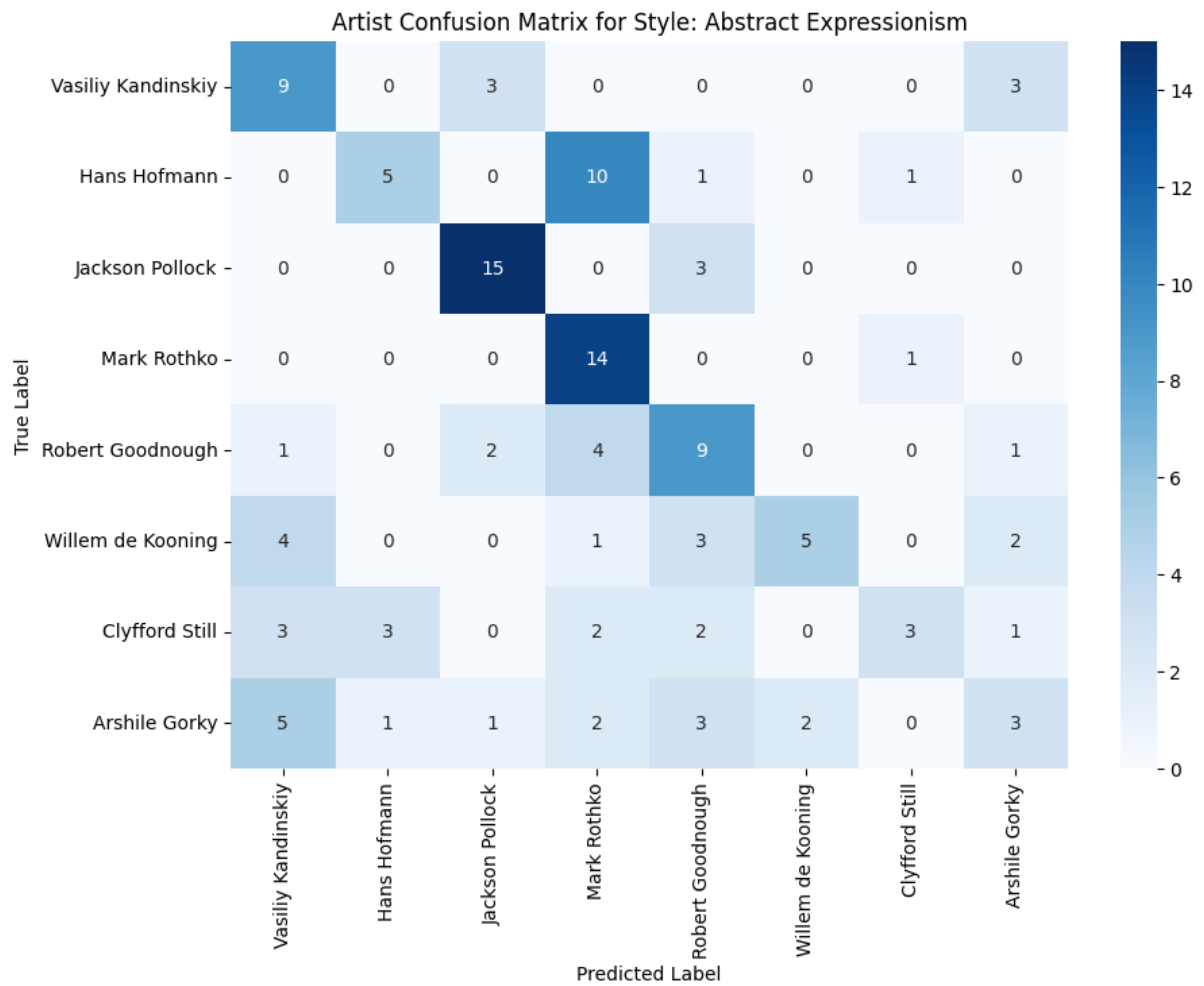


Figure 3.6.7. Confusion matrix for Abstract Expressionism for the StA model.

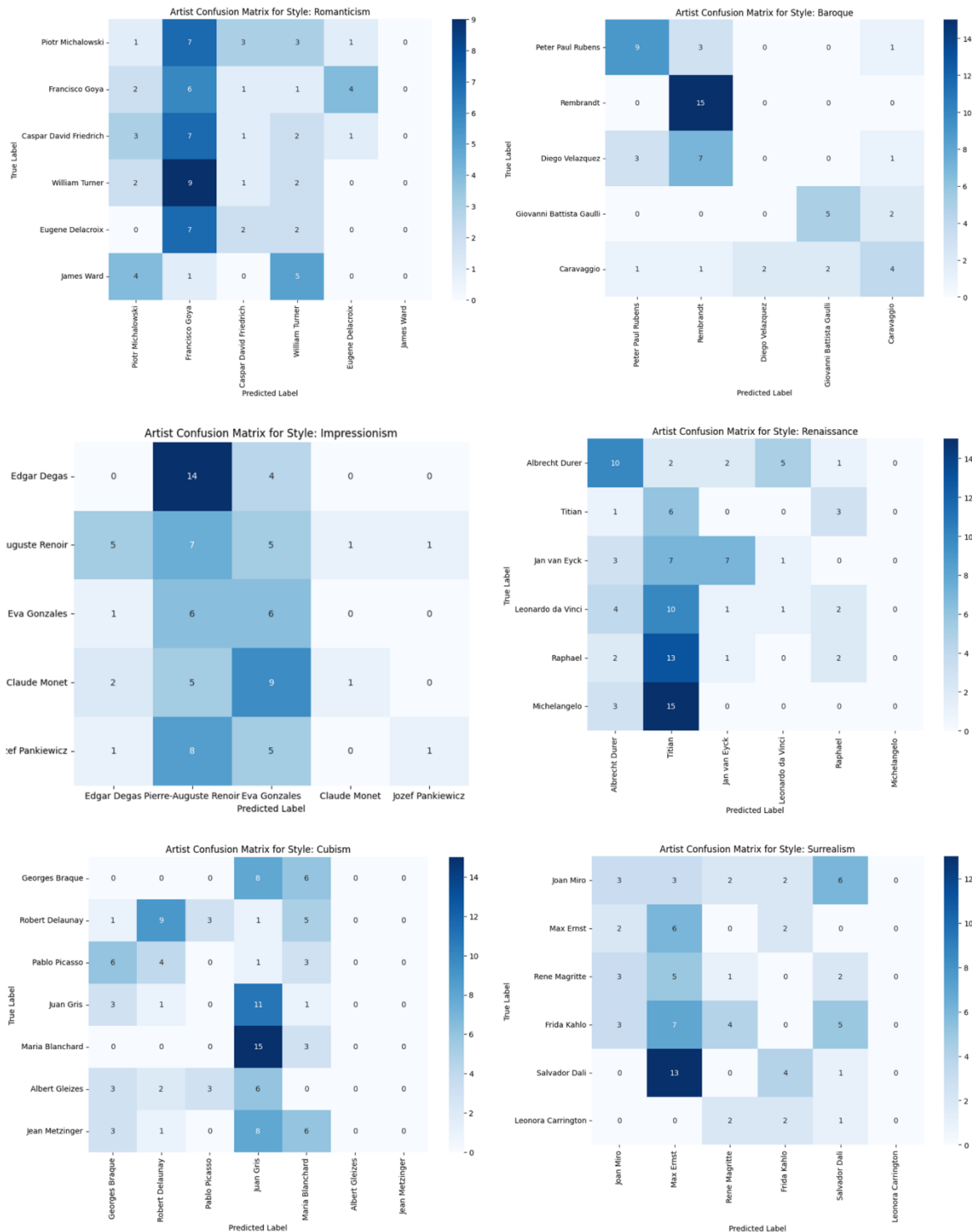


Figure 3.6.8. Confusion matrices for the remaining styles for the StA model.

Confusion matrices for artists in this case reveal a visibly larger number of misclassifications. Observations across the matrices show that certain artists are classified more accurately than others. For example, in the Impressionism matrix, Edgar Degas is predicted correctly with high accuracy, while some overlap occurs with other artists like Renoir and

Gonzales. In the Renaissance and Baroque matrices, prominent figures like Michelangelo and Rembrandt are identified correctly, but there's still some confusion with other artists within the same art movement. The Surrealism matrix shows significant accuracy for artists like Salvador Dalí and Frida Kahlo but shows some confusion among other Surrealist artists.

Overall, the model seems to perform best when predicting well-known, distinct artists that had the highest number of fragments representing their paintings in the data but struggles with those who share similar stylistic traits or are less prominent, leading to more confusion between artists within the same style.

4.4. Comparison of the models.

Metric / Model	AtS	AaS	StA
Train Artist Accuracy	87,78%	60.05%	56,94%
Train Style Accuracy	-	80.65%	77,93%
Val Artist Accuracy	61,53%	53,75%	-
Val Style Accuracy	-	74,12%	76,01%
Test Artist Accuracy	66,74%	38,95%	28.43%
Test Style Accuracy	78,84%	72,56%	74,19%
Precision	0.7279	0.6394	0.4935
Recall	0.7279	0.6394	0.4935
F1 Score	0.3640	0.3197	0.4935

Table 1. Metrics for all the created models.

Table 1 presents compiled results for all the models. The AtS model outperforms the other models in nearly every metric, particularly in artist classification accuracy. This suggests that simplifying the task by grouping artists based on their associated styles allows the model to better learn artist-specific features. It achieves high precision and recall, indicating a strong balance between identifying artists correctly and minimizing false positives.

The StA model, however, might contain the biggest potential. With extended training and more experiments with parameters and architectures, the model might prove to acquire the best results. Its advantage of narrowing the artist classification could mitigate the problem posed by the small amount of data per artist, which might have been what caused the difficulty for the AaS model for artist classification. While it initially underperforms in terms of artist accuracy

compared to AtS, its ability to narrow down the artist classification by first determining the style shows promise for optimization. The relatively higher style classification accuracy (74.19% on the test set) hints that the model is able to distinguish between styles quite well, but struggles with the artist classification aspect, likely due to the model being trained on limited artist data. Improving this model by extending training, optimizing the architecture, or employing more data augmentation strategies could help it outperform the other approaches. It is likely that with more data and fine-tuning, the StA model could excel, especially given its potential to handle the issue of insufficient artist data within each style.

The AaS model struggles the most, especially in artist classification. This might be because the simultaneous learning of two tasks (multi-label classification) adds complexity to the model. The performance drop on both artist accuracy (38.95%) and style accuracy (72.56%) indicates that the multi-label problem might confuse the model, especially since the dataset might not provide enough distinctive information for both labels in certain cases. This model might require either significantly more data or a more sophisticated architecture to balance the dual classification task effectively.

An interesting pattern is that while the AtS model shows a high training artist accuracy (87.78%), its validation and test accuracy for artists is considerably lower (61.53% and 66.74%, respectively). This difference between training and test performance suggests that the AtS model might have some overfitting issues, as was mentioned in previous chapters. The model could be learning artist-specific patterns that don't generalize well to unseen data. Similarly, AaS and StA models also exhibit signs of overfitting, but not as pronounced, as seen in the drop from training to test accuracy for artist classification in AaS (60.05% → 38.95%).

The disparity in F1 scores across the models reflects the models ability to balance precision and recall. StA, with the highest F1 score (0.3640), appears to handle class imbalances better than the other models, as it correctly identifies artists more consistently while avoiding many false positives. AaS and AtS have much lower F1 scores, especially AaS (0.3197), suggesting that they struggle with imbalanced data or lack of distinguishing information for certain artist/style pairs.

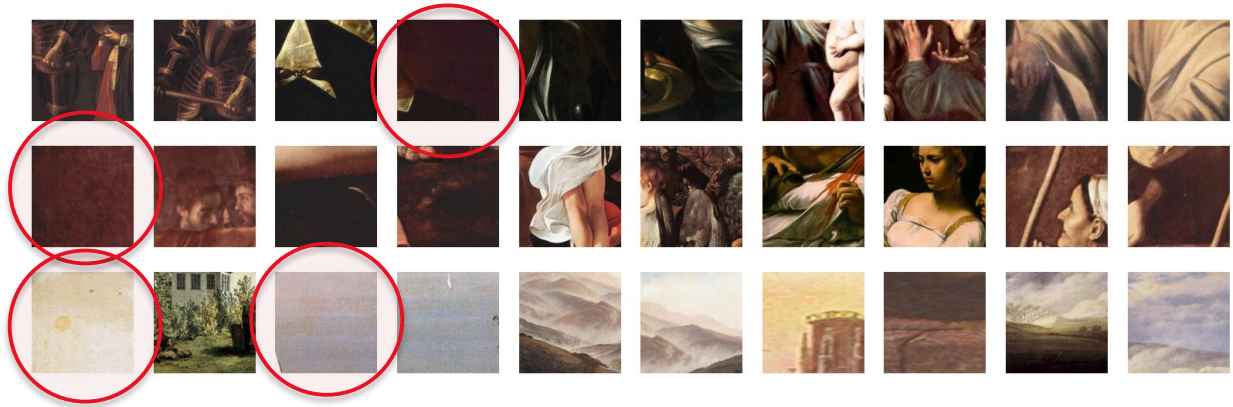


Figure 4.1.1. Examples of fragments from the dataset.

A difficulty posed for all the models is that this dataset contains multiple fragments of paintings that don't necessarily contain much information that could point the model to a specific style or artist. Examples of such fragments can be seen highlighted in Figure 4.1.1. Techniques like including the whole painting or using more sophisticated image preprocessing to extract significant features might help improve classification performance.

5. Conclusions

The goal of this thesis was to create three models to classify an author and style of fragments of paintings and compare them to each other. Each model was trained and validated to acquire the best results within the given timeframe. Transfer learning was used as well as fine-tuning and hyperparameter tuning to acquire the best results for each of the models. Then the models were compared to identify the best performing. All the goals of the thesis were reached, the models were created and compared, acquiring the results presented in the previous chapters. The results showed that the AtS model currently provides the best results, but the StA model shows significant potential for future improvement, particularly if more training and fine-tuning are applied. The AaS model struggles due to the complexity of simultaneous classification, suggesting that multi-label classification may not be the optimal approach for the given dataset.

While all the goals of the project were reached, it is important to mention potential future work and improvements that could be done to further enhance the project and broaden the exploration of the topic. First, a larger dataset could be explored with a wider range of both artists and styles. For this project's scope, their number had to be limited, so a wider set of paintings could be explored in the future. It is common for one painter to create in more than one style. The dataset used in this project assumes a simplified approach where each artist creates in only one style. In the future that could be expanded to include all styles an artist creates in.

In the first stages of this project, a few experiments regarding different methods of fragment extraction were conducted. One of the methods included object and face detection for selecting the fragments to be extracted. The experiment consisted of using both face and object detection and adding randomly selected fragments in cases where there were not enough fragments with significant faces/objects. From a few preliminary trials, the results showed a minor difference in final results between this approach and one with fragments selected completely randomly. Because of that the final version of this work uses randomly selected fragments, however, more experiments could be conducted in the future to determine whether a less random approach to fragment selection could improve the accuracy of the classification.

In the preliminary stages, some trials were made to determine if adding more augmentation could improve the accuracy of the final results. The experiments showed a very insignificant decline in accuracy, thus the augmentation methods used in the final version were

limited to the ones described earlier. In the future, more experiments could be conducted in this area to determine which augmentation methods could be more helpful in significantly increasing the accuracy for the dataset used in this project.

While in the course of working on this thesis, some variations of parameters and hyperparameters were tried, more could be tested to find out their optimal combinations for the created models.

In conclusion, the work explored the proposed models and their accuracy, as well as compared their performance on the same set of data. While there's always room for improvement the work provided an additional approach to handling art classification in a multi-label context, that takes into account both the authorship and style of paintings.

6. References

- [1] Taye, Mohammad Mustafa. "Theoretical understanding of convolutional neural network: Concepts, architectures, applications, future directions." *Computation* 11.3 (2023): 52.
- [2] Li, Zewen, et al. "A survey of convolutional neural networks: analysis, applications, and prospects." *IEEE transactions on neural networks and learning systems* 33.12 (2021): 6999-7019.
- [3] Bezdan, Timea, and N. Bačanić Džakula. "Convolutional neural network layers and architectures." *International scientific conference on information technology and data related research*. Belgrade, Serbia: Singidunum University, 2019.
- [4] Koushik, Jayanth. "Understanding convolutional neural networks." *arXiv preprint arXiv:1605.09081* (2016).
- [5] Liang, Jiefeng, et al. "Two-terminal fault location method of distribution network based on adaptive convolution neural network." *IEEE Access* 8 (2020): 54035-54043.
- [6] Pang, Yanwei, et al. "Convolution in convolution for network in network." *IEEE transactions on neural networks and learning systems* 29.5 (2017): 1587-1597.
- [7] <https://machinelearningmastery.com/cross-entropy-for-machine-learning/>
- [8] [https://medium.com/unpackai/cross-entropy-loss-in-ml-d9f22fc11fe0#:~:text=Cross%2Dentropy%20can%20be%20calculated,*%20log\(Q\(x\)\)](https://medium.com/unpackai/cross-entropy-loss-in-ml-d9f22fc11fe0#:~:text=Cross%2Dentropy%20can%20be%20calculated,*%20log(Q(x)))
- [9] Zhao, Wentao, et al. "Compare the performance of the models in art classification." *Plos one* 16.3 (2021): e0248414.
- [10] de Menezes, Richardson Santiago Teles, et al. "Classification of paintings authorship using convolutional neural network." *Sociedade Brasileira de Inteligência Computacional* (2021).
- [11] Arabie, P., et al. "Hierarchical classification." P. Arabie, L. Hubert, G. De Soete, & A. Gordon, *Clustering and classification* (1996): 65-121.
- [12] <https://www.v7labs.com/blog/transfer-learning-guide>
- [13] Guo, Yunhui, et al. "Spottune: transfer learning through adaptive fine-tuning." *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2019.
- [14] Zhuang, Fuzhen, et al. "A comprehensive survey on transfer learning." *Proceedings of the IEEE* 109.1 (2020): 43-76.
- [15] Silla, Carlos N., and Alex A. Freitas. "A survey of hierarchical classification across different application domains." *Data mining and knowledge discovery* 22 (2011): 31-72.
- [15] <https://jason-adam.github.io/resnet50/>
- [16] Naik, Azad, and Huzefa Rangwala. "Hierflat: flattened hierarchies for improving top-down hierarchical classification." *International Journal of Data Science and Analytics* 4.3 (2017): 191-208.
- [17] Yan, Zhicheng, et al. "HD-CNN: hierarchical deep convolutional neural networks for large scale visual recognition." *Proceedings of the IEEE international conference on computer vision*. 2015.

- [18] Balyan, Renu, Kathryn S. McCarthy, and Danielle S. McNamara. "Applying natural language processing and hierarchical machine learning approaches to text difficulty classification." *International Journal of Artificial Intelligence in Education* 30.3 (2020): 337-370.
- [19] Seo, Yian, and Kyung-shik Shin. "Hierarchical convolutional neural networks for fashion image classification." *Expert systems with applications* 116 (2019): 328-339.
- [20] Colonna, Juan G., João Gama, and Eduardo F. Nakamura. "Recognizing family, genus, and species of anuran using a hierarchical classification approach." *Discovery Science: 19th International Conference, DS 2016, Bari, Italy, October 19–21, 2016, Proceedings 19*. Springer International Publishing, 2016.
- [21] Huang, Meiyan, et al. "Longitudinal measurement and hierarchical classification framework for the prediction of Alzheimer's disease." *Scientific reports* 7.1 (2017): 39880.
- [22] Tsoumakas, G., Katakis, I., and Vlahavas, I. Mining multilabel data. In Maimon, O. and Rokach, L. (eds.), *Data Mining and Knowledge Discovery Handbook*, pp. 667–685. Springer, 2010.
- [23] Zhang, Min-Ling, et al. "Binary relevance for multi-label learning: an overview." *Frontiers of Computer Science* 12 (2018): 191-202.
- [24] Read, Jesse, et al. "Classifier chains: A review and perspectives." *Journal of Artificial Intelligence Research* 70 (2021): 683-718.
- [25] Spolaôr, Newton, et al. "Using label powerset for multi-label feature selection: an experimental comparison."
- [26] Bi, Wei, and James Kwok. "Efficient multi-label classification with many labels." *International conference on machine learning*. PMLR, 2013.
- [27] Chougrad, Hiba, Hamid Zouaki, and Omar Alheyane. "Multi-label transfer learning for the early diagnosis of breast cancer." *Neurocomputing* 392 (2020): 168-180.
- [28] Mao, Hui, Ming Cheung, and James She. "Deepart: Learning joint representations of visual arts." *Proceedings of the 25th ACM international conference on Multimedia*. 2017.
- [29] Lecoutre, Adrian, Benjamin Negrevergne, and Florian Yger. "Recognizing art style automatically in painting with deep learning." *Asian conference on machine learning*. PMLR, 2017.
- [30] <https://www.wikiart.org/>
- [31] Zhao, Wentao, et al. "How to represent paintings: A painting classification using artistic comments." *Sensors* 21.6 (2021): 1940.
- [32] Liu, Shao, et al. "Novel features for art movement classification of portrait paintings." *Image and Vision Computing* 108 (2021): 104121.
- [33] Yang, Zihan. "Classification of picture art style based on VGGNET." *Journal of Physics: Conference Series*. Vol. 1774. No. 1. IOP Publishing, 2021.
- [34] Hua, Kai-Lung, et al. "Artist-based painting classification using Markov random fields with convolution neural network." *Multimedia Tools and Applications* 79 (2020): 12635-12658.

- [35] de Menezes, Richardson Santiago Teles, et al. "Classification of paintings authorship using convolutional neural network." *Sociedade Brasileira de Inteligência Computacional* (2021).
- [36] Narvaez, Marcelo Fraile, Ismael Sagredo Olivenza, and Nadia McGowan. "Painting authorship and forgery detection challenges with AI image generation algorithms: Rembrandt and 17th century Dutch painters as a case study." *IJIMAI* 7.7 (2022): 7-13.
- [37] Chiarella, Salvatore G., et al. "Investigating the negative bias towards artificial intelligence: Effects of prior assignment of AI-authorship on the aesthetic appreciation of abstract paintings." *Computers in Human Behavior* 137 (2022): 107406.
- [38] Hossin, Mohammad, and Md Nasir Sulaiman. "A review on evaluation metrics for data classification evaluations." *International journal of data mining & knowledge management process* 5.2 (2015): 1.
- [39] Shalev-Shwartz, Shai, and Shai Ben-David. *Understanding machine learning: From theory to algorithms*. Cambridge university press, 2014.
- [40] Vujović, Ž. "Classification model evaluation metrics." *International Journal of Advanced Computer Science and Applications* 12.6 (2021): 599-606.
- [41] Yacouby, Reda, and Dustin Axman. "Probabilistic extension of precision, recall, and f1 score for more thorough evaluation of classification models." *Proceedings of the first workshop on evaluation and comparison of NLP systems*. 2020.
- [42] Kosmopoulos, Aris, et al. "Evaluation measures for hierarchical classification: a unified view and novel approaches." *Data Mining and Knowledge Discovery* 29 (2015): 820-865.
- [43] Goodfellow, Ian. *Deep learning*. Vol. 196. MIT press, 2016.
- [44] Berrar, Daniel. "Cross-validation." (2019): 542-545.
- [45] Prechelt, Lutz. "Early stopping-but when?." *Neural Networks: Tricks of the trade*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2002. 55-69.
- [46] <https://www.kaggle.com/datasets/ikarus777/best-artworks-of-all-time>
- [47] <https://www.wikiart.org/>
- [48] <https://commons.wikimedia.org/>
- [49] <https://www.arthistoryproject.com/about/>
- [50] <https://totallyhistory.com/>
- [51] Tan, Wei Ren, et al. "Ceci n'est pas une pipe: A deep convolutional network for fine-art paintings classification." *2016 IEEE international conference on image processing (ICIP)*. IEEE, 2016.
- [52] <https://alumentations.ai/docs/>
- [53] <https://pytorch.org/vision/main/generated/torchvision.transforms.Normalize.html>
- [54] <https://pytorch.org/vision/main/generated/torchvision.transforms.ToTensor.html>